大学信息技术基础

学习笔记

作者：张斌

组织：河北农业大学　理工系

时间：September 22, 2024

内容收集于网络

目录

1 Excel 1

1.1 文本类型数据的输入 1
1.2 自动填充 2
1.3 手动输入公式 3
1.4 使用公式函数 3
1.5 引用 3
1.6 if 函数的基本使用 4
1.7 多条件判断 4
1.8 if 函数的嵌套 4
1.9 ifs 函数（Microsoft 365、Excel 2016 及更高版本） 5
1.10 countif 函数 5
1.11 countifs 函数（Microsoft 365、Excel 2016 及更高版本） 5
1.12 SUMIF 函数 5
1.13 SUMIFS 函数 6
1.14 LOOKUP 函数 6
1.15 文本函数 7
1.16 日期函数 9
1.17 自动筛选 10
1.18 高级筛选 11
1.19 分类汇总 12
1.20 数据透视表 13
1.21 图表 13

2 PowerPoint 16

2.1 更改字体 16
2.2 幻灯片母版 17
2.3 动画效果 17

3 智慧农业系统 19

3.1 计算机 19
3.2 计算机中的信息表示 20
3.3 字符编码 24
3.4 声音、图像、视频编码 25

4 农业数字化技术 26

5 数据传输技术 27

5.1 互联网 27
5.2 网址是什么？ 29
5.3 IP 地址 30
5.4 DNS 域名系统 32

5.5 互联网的三种文档 33
5.6 计算机网络体系结构 34
5.7 协议端口 37
5.8 TCP/IP 多路复用 38
5.9 三次握手 38
5.10 四次挥手 39
5.11 TCP 可靠通信的具体实现 39
5.12 TCP 报文段首部格式 40
5.13 UDP 的首部格式 41
5.14 ARP 地址解析协议 42
5.15 链路层 42
5.16 虚拟专用网 VPN 43

6 计算思维与算法 45
6.1 什么是计算思维？ 45
6.2 计算思维的本质 46
6.3 计算思维对其他学科的影响 48
6.4 算法的表示 48
6.5 迭代算法 49
6.6 递归算法 50
6.7 分治算法 52
6.8 贪心算法 53
6.9 动态规划 55
6.10 回溯法 59
6.11 算法实例 60

第 1 章 Excel

1.1 文本类型数据的输入 1
1.2 自动填充 2
1.3 手动输入公式 3
1.4 使用公式函数 3
1.5 引用 3
1.6 if 函数的基本使用 4
1.7 多条件判断 4
1.8 if 函数的嵌套 4
1.9 ifs 函数（Microsoft 365、Excel 2016 及更高版本） 5
1.10 countif 函数 5
1.11 countifs 函数（Microsoft 365、Excel 2016 及更高版本） 5
1.12 SUMIF 函数 5
1.13 SUMIFS 函数 6
1.14 LOOKUP 函数 6
1.14.1 VLOOKUP 函数 6
1.14.2 HLOOKUP 函数 6
1.14.3 XLOOKUP 函数 7
1.15 文本函数 7
1.15.1 LEN 函数 7
1.15.2 LENB 函数 7
1.15.3 LEFT、LEFTB 函数 8
1.15.4 TEXTJOIN 函数 8
1.15.5 TEXTSPLIT 函数 8
1.16 日期函数 9
1.16.1 YEAR、MONTH 和 DAY 函数 9
1.16.2 TODAY 函数 9
1.16.3 DATE 函数 9
1.16.4 更改日期格式 10
1.16.5 时间函数 10
1.17 自动筛选 10
1.18 高级筛选 11
1.19 分类汇总 12
1.20 数据透视表 13
1.21 图表 13

1.1 文本类型数据的输入

Excel 中有三类数据类型

1. 数值
2. 文本

3. 公式

数值默认右对齐，文本默认左对齐。

编号	姓名	学号
01	张钧烁	2022984150101
02	张坪	2022984150102
03	展李戎	2022984150103
04	曹骏	2022984150104
05	朱高风	2022984150105

首位是 0 的数字，0 不显示？

位数很多的数字，自动转换成科学计数法？

解决办法：

1. 设置单元格格式为文本型
2. 数字前加单引号

1.2 自动填充

数值型单元格，拖动默认进行单元格复制；文本型单元格，如果有数字，拖动默认填充序列。

121	121
122	121
123	121
124	121
125	121

1. 通过自动填充选项按钮修改
2. 按住 Ctrl 键 + 鼠标左键下拉，切换模式
3. 按住鼠标右键下拉，填充序列
4. 填写多个单元格后，下拉

office2019 及之后版本没有自动填充选项按钮？

点击“文件”-“选项”-“高级”-“剪贴，复制和粘贴”下的“粘贴内容时显示粘贴选项按钮”点击前面的方框，勾选。

Excel 中可自动填充的序列有：

阳历月份	农历月份	星期		天干	地支	英文月份		英文星期	
一月	正月	星期一	一	甲	子	Jan	January	Mon	Monday
二月	二月	星期二	二	乙	丑	Feb	February	Tue	Tuesday
三月	三月	星期三	三	丙	寅	Mar	March	Wed	Wednesday
四月	四月	星期四	四	丁	卯	Apr	April	Thu	Thursday
五月	五月	星期五	五	戊	辰	May	May	Fri	Friday
六月	六月	星期六	六	己	巳	Jun	June	Sat	Saturday
七月	七月	星期日	日	庚	午	Jul	July	Sun	Sunday
八月	八月	星期一	一	辛	未	Aug	August	Mon	Monday
九月	九月	星期二	二	壬	申	Sep	September	Tue	Tuesday
十月	十月	星期三	三	癸	酉	Oct	October	Wed	Wednesday
十一月	十一月	星期四	四	甲	戌	Nov	November	Thu	Thursday
十二月	腊月	星期五	五	乙	亥	Dec	December	Fri	Friday

1.3 手动输入公式

运算符类型	算符	说明
数学运算符	+、－、*、/、%	加、减、乘、除、乘幂、百分比
负号	-	负号
引用运算符	冒号:、单个空格、逗号,	引用公式范围、分隔作用
文本运算符	&	连接两个文本字符串
比较运算符	=、<>、<、<=、>、>=	等于、不等于、小于、 小于等于、大于、大于等于

1. 引用单元格计算
2. 直接计算

1.4 使用公式函数

函数名 (函数参数，函数参数，…，函数参数) = 返回值

聚合函数：

- SUM 求和
- AVERAGE 求平均值
- COUNT (只对数字单元格计数) COUNTA(数字文本全计数)
- MAX 求最大值
- MIN 求最小值

1.5 引用

- 相对引用：对于 A 列第一行的单元格，相对引用是 A1。
- 绝对引用：列号和行号前面都加了一个美元的符号 A1。
- 混合引用：在行号或者列号其中一个前面加了美元符号，A$1 或 $A1，表示只固定行或者列。

1.6 if 函数的基本使用

IF 函数是 Excel 中最常用的函数之一，它可以对值和期待值进行逻辑比较。

使用逻辑函数 IF 函数时，如果条件为真，该函数将返回一个值；如果条件为假，函数将返回另一个值。

IF 函数有三个参数，语法如下：

=IF(logical_test, value_if_true, [value_if_false])

=IF(条件判断, 结果为真返回值, 结果为假返回值)

第一参数是条件判断，比如说 A1=“张三” 或 “A2>500” 这种，结果返回 TRUE 或 FALSE。如果判断返回 TRUE 那么 IF 函数返回值是第二参数，否则返回第三参数。

- 第一参数在某些情况下可以不用比较判断符号来判断。Excel 中将 0 认为是 FALSE，非 0 值等价于 TRUE。

  =IF(A1,“真”,“假”)

  如果 A1 为非零值，返回 “真”，如果 A1 为 0，返回 “假”。
- 省略了第三参数，则返回第三参数值时会返回 FALSE。

  =IF(1>2,“真”)

  返回 FALSE
- 逗号后面不输入参数默认返回值为 0。

  =IF(1>2,“真”,)

  返回 0
- 如果要返回空白，引号内留空。

  =IF(1>2,“真”,“”)
- 如果要在公式中使用文本，需要将文字用引号括起来（例如 “Text”）。唯一的例外是使用 TRUE 和 FALSE 时，Excel 能自动理解它们。

1.7 多条件判断

- 多条件同时满足

  =IF(AND(A2>29,B2=“A”),“优秀”,“”)

  这里用 AND() 函数表达了多个判断条件，只有所有判断都为 TRUE 才返回 “优秀”。
- 多条件满足其一

  =IF(OR(A2=“A”,B2>100),“合格”,“不合格”)

  用 OR() 函数来表达满足条件之一，也就是 OR() 函数内的判断只要有一个返回 TRUE，OR() 函数整体就返回 TRUE。

1.8 if 函数的嵌套

=IF(D2>89,”A”,IF(D2>79,”B”,IF(D2>69,”C”,IF(D2>59,”D”,”F”))))

如果单元格 D2 大于 89，则学生获得 A

如果单元格 D2 大于 79，则学生获得 B

如果单元格 D2 大于 69，则学生获得 C

如果单元格 D2 大于 59，则学生获得 D

否则，学生获得 F

- 为了使长公式更易于阅读，可在编辑栏中插入换行符。只需在将文本换到新行前按 Alt+Enter。

1.9 ifs 函数（Microsoft 365、Excel 2016 及更高版本）

IFS 函数检查是否满足一个或多个条件，且返回符合第一个 TRUE 条件的值。IFS 可以取代多个嵌套 IF 语句，并且有多个条件时更方便阅读。

通常情况下，IFS 函数的语法如下：

=IFS(logical_test1,value_if_true1,logical_test2,value_if_true2,logical_test3,value_if_true3)

=IFS(A2>89,”A”,A2>79,”B”,A2>69,”C”,A2>59,”D”,TRUE,”F”)

如果（A2 大于 89，则返回 “A”，如果 A2 大于 79，则返回 “B” 并以此类推，对于所有小于 59 的值，返回 “F”）

- 若要指定默认结果，请对最后一个 logical_test 参数输入 TRUE。如果不满足其他任何条件，则将返回相应值。
- 如果提供了 logical_test 参数，但未提供相应的 value_if_true，则此函数显示 “你为此函数输入的参数过少” 错误消息。
- 如果 logical_test 参数经计算解析为 TRUE 或 FALSE 以外的值，则此函数返回 #VALUE! 错误。
- 如果找不到 TRUE 条件，则此函数返回 #N/A! 错误。

1.10 countif 函数

计算某个区域中满足给定条件的单元格数目

=COUNTIF(要检查哪些区域? 要查找哪些内容?)

例如：

=COUNTIF(A2:A5,“London”)

=COUNTIF(A2:A5,A4)

- 任何文本条件或任何含有逻辑或数学符号的条件都必须使用双引号 (“ ”) 括起来。如果条件为数字或单元格引用，则无需使用双引号。
- criteria 不区分大小写。
- 可以在 criteria 中使用通配符，即问号 (?) 和星号 (*)。

1.11 countifs 函数（Microsoft 365、Excel 2016 及更高版本）

计算某个区域中满足多个条件的单元格数目

语法

COUNTIFS(criteria_range1, criteria1, [criteria_range2, criteria2],…)

- criteria_range1 必需。在其中计算关联条件的第一个区域。
- criteria1 必需。条件的形式为数字、表达式、单元格引用或文本，它定义了要计数的单元格范围。例如，条件可以表示为 32、”>32”、B4、”apples” 或”32”。
- criteria_range2, criteria2, ... 可选。附加的区域及其关联条件。
- 每一个附加的区域都必须与参数 criteria_range1 具有相同的行数和列数。这些区域无需彼此相邻。
- 如果条件参数是对空单元格的引用，COUNTIFS 会将该单元格的值视为 0。
- 可以在条件中使用通配符，即问号 (?) 和星号 (*)。如果要查找实际的问号或星号，请在字符前键入波形符 (~)。

1.12 SUMIF 函数

可以使用 SUMIF 函数对范围中符合指定条件的值求和。

语法

SUMIF(range, criteria, [sum_range])

- range 范围，必需。希望通过标准评估的单元格范围。
- criteria 必需。criteria 以数字、表达式、单元格参考、文本或函数的形式来定义将添加哪些单元格。
  例如，criteria 可以表示为 32、“>32”、B5、“3?”、“苹果 *” 或今天 ()。
- sum_range 可选。要求和的单元格区域，如果要在 range 参数指定以外的其他单元格求和。sum_range 的大小和形状应该与 range 相同。

例如，如果某列中含有数字，你需要对大于 5 的数值求和。可使用以下公式：

=SUMIF(B2:B25,”>5”)

如果需要，可将条件应用于一个区域并对其他区域中的对应值求和。

例如，公式 =SUMIF(B2:B5, ”John”, C2:C5) 只对区域 C2:C5 中在区域 B2:B5 中所对应的单元格等于“John”的值求和。

1.13 SUMIFS 函数

SUMIFS 函数用于计算满足多个条件的全部参数的总量。

语法 SUMIFS(sum_range, criteria_range1, criteria1, [criteria_range2, criteria2], ...)

=SUMIFS(A2:A9,B2:B9,”<> 香蕉”,C2:C9,” 卢宁”)

- Sum_range （必需）要求和的单元格区域。
- Criteria_range1 （必需）使用 Criteria1 测试的区域。
- Criteria1 （必需）定义将计算 Criteria_range1 中的哪些单元格的和的条件。
- Criteria_range2, criteria2, …（可选）附加的区域及其关联条件。

1.14 LOOKUP 函数

1.14.1 VLOOKUP 函数

需要在表格或区域中按行查找内容时，请使用 VLOOKUP。

语法：

=VLOOKUP(要查找的内容、要查找的位置、包含要返回的值的范围内的列号、返回表示为 1/TRUE 或 0/FALSE 的近似或精确匹配项)。

- 查阅值应该始终位于所在区域的第一列，这样 VLOOKUP 才能正常工作。
  例如，如果查阅值位于单元格 C2 内，那么您的区域应该以 C 开头。
- 返回值的列号：如果指定 B2:D11 作为区域，那么应该将 B 算作第一列，C 作为第二列，以此类推。
- 使用 VLOOKUP 将多个表中的数据合并到一个工作表上
  只要其中一个表具有与所有其他表共有的字段，就可以使用 VLOOKUP 将多个表合并为一个表。

1.14.2 HLOOKUP 函数

查找数组的首行，并返回指定单元格的值。

语法

HLOOKUP(lookup_value, table_array, row_index_num, [range_lookup])

- Lookup_value 必需。要在表格的第一行中查找的值。Lookup_value 可以是数值、引用或文本字符串。
- Table_array 必需。在其中查找数据的信息表。使用对区域或区域名称的引用。

- Row_index_num 必需。行号 table_array 匹配值将返回的行号。
- Range_lookup 可选。一个逻辑值，指定希望 HLOOKUP 查找精确匹配值还是近似匹配值。如果为 TRUE 或省略，则返回近似匹配值。

1.14.3 XLOOKUP 函数

XLookup 函数是微软在 2019 年 8 月份发布的一个最新的查找函数。

XLOOKUP 函数搜索区域或数组，然后返回与它找到的第一个匹配项对应的项。如果不存在匹配项，则 XLOOKUP 可以返回最接近 () 匹配项。

语法

=XLOOKUP(lookup_value, lookup_array, return_array, [if_not_found], [match_mode], [search_mode])

- lookup_value （必需）要搜索的
- lookup_array （必需）要搜索的数组或区域
- return_array （必需）要返回的数组或区域
- [if_not_found]（可选）如果找不到有效的匹配项，请返回你提供的 [if_not_found] 文本。
  如果找不到有效的匹配项，并且 [if_not_found] 缺失，则返回 #N/A 。
- [match_mode]（可选）指定匹配类型：
  - 0 - 完全匹配。如果未找到，则返回 #N/A。这是默认选项。
  - -1 - 完全匹配。如果没有找到，则返回下一个较小的项。
  - 1 - 完全匹配。如果没有找到，则返回下一个较大的项。
  - 2 - 通配符匹配。
- [search_mode]（可选）指定要使用的搜索模式：
  - 1 - 从第一项开始执行搜索。这是默认选项。
  - -1 - 从最后一项开始执行反向搜索。
  - 2 - 执行依赖于 lookup_array 按升序排序的二进制搜索。如果未排序，将返回无效结果。
  - 2 - 执行依赖于 lookup_array 按降序排序的二进制搜索。如果未排序，将返回无效结果。

1.15 文本函数

1.15.1 LEN 函数

这个函数的主要功能，就是计算单元格中的字符数。

使用 LEN 函数计算字符数的时候，空格也是会被算作一个字符。

```
A2 = 苹果apple
LEN(A2) = 7
```

1.15.2 LENB 函数

LENB 函数的主要作用，是计算单元格中的字节数量。

LENB 函数的计算结果大于 LEN 函数的计算结果，单元格中的所有汉字都被视为两个字节。

```
    A2 = 苹果apple
    LENB(A2) = 9
```

1.15.3 LEFT、LEFTB 函数

LEFT 函数的主要功能，就是从左边开始返回特定数量的字符。

LEFTB 基于所指定的字节数返回文本字符串中的第一个或前几个字符。

- Left 函数把全角（如汉字）和半角字符（如数字与字母）都计作一个字符
- LeftB 函数把全角字符计作两个字节、半角字符计作一个字节

语法：

LEFT(text, [num_chars])

LEFTB(text, [num_bytes])

```
A2 = 苹果apple
LEFT(A2,2) = 苹果
LEFT(A2,LENB(A2)-LEN(A2)) = 苹果
```

结合以上 3 个函数，可以实现将中英文字符分离。

1.15.4 TEXTJOIN 函数

TEXTJOIN 函数将多个区域和/或字符串的文本组合起来，并包括你在要组合的各文本值之间指定的分隔符。如果分隔符是空的文本字符串，则此函数将有效连接这些区域。

语法：TEXTJOIN(delimiter, ignore_empty, text1, [text2], …)

- delimiter：分隔符 (必需)，文本字符串，或者为空，或用双引号引起来的一个或多个字符，或对有效文本字符串的引用。如果提供一个数字，则将被视为文本。
- ignore_empty (必需)，如果为 TRUE，则忽略空白单元格。
- text1(必需)，要联接的文本项。文本字符串或字符串数组，如单元格区域中。
- [text2, ...](可选)，要联接的其他文本项。文本项最多可以包含 252 个文本参数 text1。每个参数可以是一个文本字符串或字符串数组，如单元格区域。

例如，=TEXTJOIN(" ",TRUE,"The","sun","will","come","up","tomorrow") 将返回 The sun will come up tomorrow

1.15.5 TEXTSPLIT 函数

使用列和行分隔符拆分文本字符串。

TEXTSPLIT 函数的工作方式与文本转列向导相同，但采用公式形式。它允许跨列拆分或按行向下拆分。它是 TEXTJOIN 函数的反函数。

语法：

=TEXTSPLIT(text,col_delimiter,[row_delimiter],[ignore_empty],

[match_mode], [pad_with])

TEXTSPLIT 函数语法具有下列参数：

- text 要拆分的文本。必需。
- col_delimiter 标记跨列溢出文本的点的文本。
- row_delimiter 标记向下溢出文本行的点的文本。可选。
- ignore_empty 指定 TRUE 以忽略连续分隔符。默认为 FALSE，将创建一个空单元格。可选。
- match_mode 指定 1 以执行不区分大小写的匹配。默认为 0，这会执行区分大小写的匹配。可选。
- pad_with 用于填充结果的值。默认值为 #N/A。

1.16 日期函数

1.16.1 YEAR、MONTH 和 DAY 函数

YEAR：返回对应于某个日期的年份。Year 作为 1900 - 9999 之间的整数返回。

MONTH：返回日期中的月份。月份是介于 1（一月）到 12（十二月）之间的整数。

DAY：返回以序列数表示的某日期的天数。天数是介于 1 到 31 之间的整数。

=YEAR(A3)

=YEAR(“2022/11/14”)

=YEAR(“2022-11-14”)

- 当直接在函数中输入日期时，您需要将其用引号引起。
- Microsoft Excel 可将日期存储为可用于计算的序列号。默认情况下，1900 年 1 月 1 日的序列号是 1，而 2008 年 1 月 1 日的序列号是 39448，这是因为它距 1900 年 1 月 1 日有 39448 天。
- 无论提供的日期值的显示格式如何，YEAR、MONTH 和 DAY 函数返回的值都是公历值。例如，如果提供的日期的显示格式是农历，则 YEAR、MONTH 和 DAY 函数返回的值将是与对应的公历日期相关联的值。

1.16.2 TODAY 函数

返回当前日期的序列号。

语法：

TODAY()

TODAY 函数语法没有参数。

不管您何时打开工作薄，当需要在工作表上显示当前日期时，TODAY 函数非常有用。

它还可用于计算时间间隔。例如，如果您知道某人出生于 1963 年，您可使用以下公式计算对方到其今年生日为止的年龄：

= YEAR(TODAY())-1963

1.16.3 DATE 函数

如果需要采用三个单独的值并将它们合并为一个日期，请使用 DATE 函数。

语法：DATE(year,month,day)

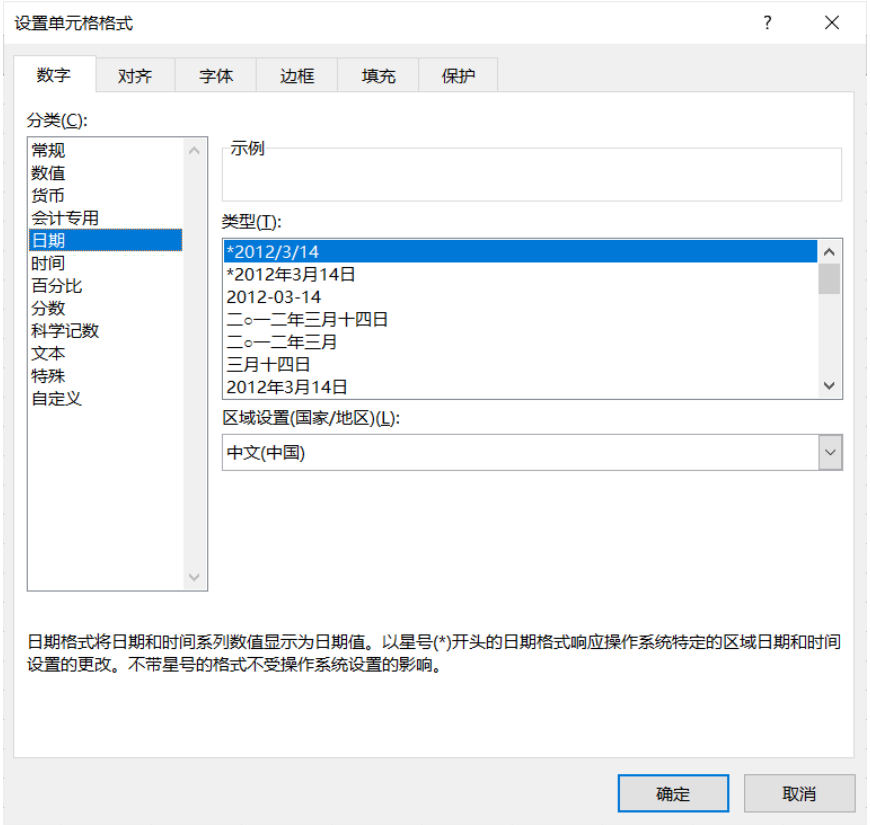
- Year 必需。默认情况下，Excel 使用 1900 日期系统，这意味着第一个日期是 1900 年 1 月 1 日。
  - 如果 year 介于 0（零）到 1899 之间（包含这两个值），则 Excel 会将该值与 1900 相加来计算年份。例如，DATE(108,1,2) 返回 2008 年 1 月 2 日 (1900+108)。
  - 如果 year 介于 1900 到 9999 之间（包含这两个值），则 Excel 将使用该数值作为年份。
  - 如果 year 小于 0 或大于 10000，Excel 将返回 #NUM！错误值。
- Month 必需。
  - 如果 month 大于 12，则 month 会从指定年份的第一个月开始加上该月份数。例如，DATE(2008,14,2) 返回表示 2009 年 2 月 2 日的序列数。
  - 如果 month 小于 1，则 month 会从指定年份的第一个月开始减去该月份数，然后再加上 1 个月。例如，DATE(2008,-3,2) 返回表示 2007 年 9 月 2 日的序列号。
- Day 必需。
  - 如果 day 大于指定月中的天数，则 day 会从该月的第一天开始加上该天数。例如，DATE(2008,1,35) 返回表示 2008 年 2 月 4 日的序列数。

- 如果 day 小于 1，则 day 从指定月份的第一天开始减去该天数，然后再加上 1 天。例如，DATE(2008,1,-15) 返回表示 2007 年 12 月 16 日的序列号。

1.16.4 更改日期格式

当您在单元格中输入一些文本（如 “2/2”）时，Excel 会假定这是日期，并且根据 “控制面板” 中的默认日期设置设置格式。

更改日期格式：右键单击要更改的单元格，设置单元格格式或者按 Ctrl+1



单元格中的数字是否显示为 #####？单元格可能不够宽，无法显示整个数字。请尝试双击含 ##### 单元格的列的右边界。这样可以重新调整列的大小，使其适应数字。您也可以拖动列的右边界，以使列达到所需的大小。

1.16.5 时间函数

HOUR：返回时间的小时数，小时数是介于 0 到 23 之间的整数。

MINUTE：返回时间的分钟数，分钟是一个介于 0 到 59 之间的整数。

SECOND：返回时间的秒数，秒数是 0 到 59 范围内的整数。

Now()：返回当前日期和时间的序列号。NOW 函数语法没有参数。

TIME：返回特定时间的十进制数字。如果在输入该函数之前单元格格式为 “常规”，则结果将使用日期格式。

TIME(hour, minute, second)

Excel 将每 24 小时的周期作为一个从 0 至 1（不包括 1）的小数存储。

由 TIME 返回的十进制数字是一个范围在 0（零）到 0.999988426 之间的值，表示 0:00:00 (12:00:00 AM) 到 23:59:59 (11:59:59 P.M.) 之间的时间。

1.17 自动筛选

筛选区域或表中的数据：

1. 选择区域内的任意单元格。
2. 选择“数据 > 筛选器”。
3. 选择列标题箭头
4. 选择“文本筛选器”或“数字筛选器”，然后选择条件
5. 输入筛选条件，然后选择“确定”。

清除或删除筛选器：

- 清除列中的筛选器

  单击列标题“筛选”按钮，然后单击列名称中的“清除”。
- 清除所有筛选器

  单击“数据”选项卡，然后单击“清除”。
- 删除所有筛选器

  单击“数据”选项卡，然后单击“筛选”按钮

我如何知道数据是否已经过筛选？

筛选器可用，并且尚未用于对列中的数据进行排序。

数据已经过筛选。

1.18 高级筛选

如果要筛选的数据需要复杂条件（例如类型 = “农产品”或销售人员 = “李小明”），可使用“高级筛选”对话框。

若要打开“高级筛选”对话框，请单击“数据 > 高级”。

可以在工作表以及要筛选的单元格区域或表格上的单独条件区域中键入高级条件。

使用等号键入文本或值

由于在单元格中键入文本或值时等号 (=) 用来表示一个公式，因此 Excel 会计算键入的内容；不过，这可能会产生意外的筛选结果。为了表示文本或值的相等比较运算符，应在条件区域的相应单元格中键入作为字符串表达式的条件：

= “= 条目”

其中条目是要查找的文本或值。

多列中有多个条件，所有条件都为真

例：(类型 = “农产品” AND 销售人员 = “林恪”)

若要查找满足多列中有多个条件的行，请在条件区域的同一行中键入所有条件。

多列中有多个条件，任何条件为真

例：(类型 = “农产品” OR 销售人员 = “林恪”)

若要查找满足多列中有多个条件，其中任何条件都可以为真的行，请在条件区域的不同列和行中键入条件。

多组条件，所有组集中于一列中

例：((销售额 > 6000 AND 销售额 < 6500) OR (销售额 < 500))

若要查找满足多组条件，每组包含用于一列的条件的行，请在多个列中包括同一个列标题。

多组条件，每组包含多列

例：((销售人员 = “赵强” AND 销售额 >3000) OR (销售人员 = “林恪” AND 销售额 > 1500))

若要查找满足多组条件，其中每组包括用于多列的条件的行，请在单独的列和行中键入每组条件。

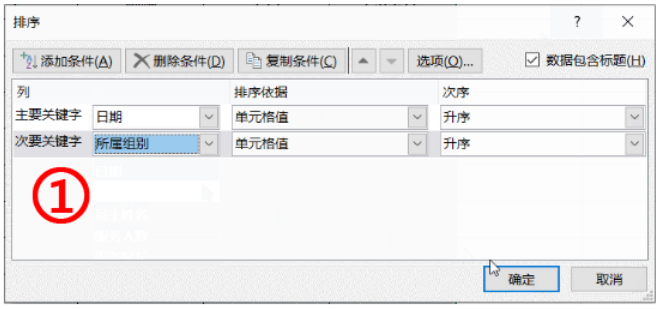
1.19 分类汇总

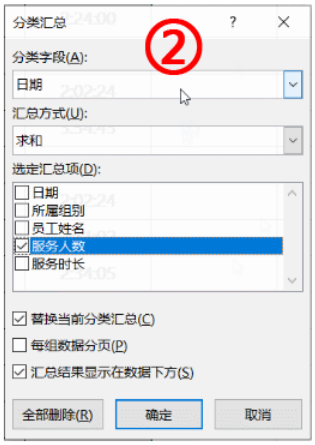
在表格中实现简单的统计分析工作，相信大家第一时间肯定会想到使用函数公式，或者是数据透视表。其实，Excel 中还有一个非常实用的功能，能帮助你快速实现表格的多级统计，它的名字叫分类汇总。

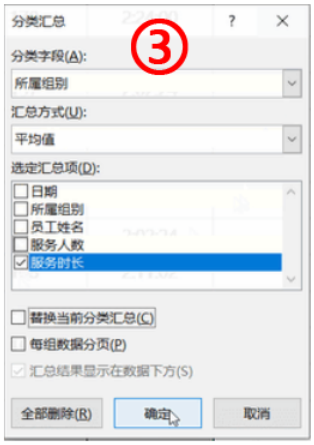
接下来，我们就来学习分类汇总的使用方法。

看下面这个案例，需要快速统计出每天的服务人数总和，以及每天每个组别的平均服务时长，我们可以使用分类汇总功能来实现。

日期	所属组别	员工姓名	服务人数	服务时长
8月1日	1	张志山	111	3:14:24
8月1日	2	王卫平	100	3:43:12
8月1日	1	陈鹏	144	2:05:17
8月1日	3	高琳	197	2:02:24
8月1日	1	黄璐京	170	2:24:00
8月1日	3	杨宝春	125	3:54:43
8月2日	1	张志山	144	2:58:34
8月2日	1	陈鹏	192	2:02:24
8月2日	1	黄璐京	108	2:11:02
8月2日	3	王川	137	3:53:17
8月2日	1	刘学燕	182	2:34:05
8月2日	3	高琳	104	2:58:34
8月2日	2	张晓寰	165	3:12:58
8月3日	1	黄璐京	187	3:21:36
8月3日	3	杨宝春	140	2:26:53
8月3日	1	陈鹏	197	3:11:31
8月3日	2	许东东	126	3:12:58







1. 先对汇总条件进行排序

   统计的条件有两个，分别是日期和组别，我们使用自定义排序功能，添加排序条件，主要关键字选择【日期】，次序选择【升序】；次要关键字选择【所属组别】，次序选择【升序】，完成对数据的排序。

2. 按日期为分类做一级汇总

   点击打开【数据】选项卡，使用【分类汇总】功能，分类字段选择【日期】，选定汇总项勾选【服务人数】，做出一级统计。

3. 按组别为分类做二级汇总

   再次使用分类汇总功能，分类字段选择【所属组别】，汇总方式选择【平均值】，选定汇总项勾选【服务时长】。这里需要注意的是，要把【替换当前分类汇总】的勾选去掉，才能实现二级汇总。

实现分类汇总后，我们可以从表格中快速地看出每天每个组别的平均服务时长，以及每天的服务人数。

点击左侧的加减号或者组别，能够折叠和展开每个层级的数据，方便查看明细数据和汇总数据。

最后，如果要删除分类汇总，重新打开分类汇总功能，点击全部删除即可。

备注：数据分类汇总与数据透视表相比，数据透视表相对灵活方便，建议实际分类汇总数据中使用数据透视表。

A	B	C	D	E
日期	所属组别	员工姓名	服务人数	服务时长
8月1日	1	张志山	111	3:14:24
8月1日	1	陈鹏	144	2:05:17
8月1日	1	黄璐京	170	2:24:00
	1 平均值			2:34:34
8月1日	2	王卫平	100	3:43:12
	2 平均值			3:43:12
8月1日	3	高琳	197	2:02:24
8月1日	3	杨宝春	125	3:54:43
	3 平均值			2:58:34
8月1日 汇总			847	
8月2日	1	张志山	144	2:58:34
8月2日	1	陈鹏	192	2:02:24
8月2日	1	黄璐京	108	2:11:02
8月2日	1	刘学燕	182	2:34:05
	1 平均值			2:26:31
8月2日	2	张晓寰	165	3:12:58

1.20 数据透视表

数据透视表建立在明细数据记录基础之上。集合了排序、筛选和分类汇总的功能，并生成汇总表格，集中体现了 Excel 强大的数据分析功能。

1. 创建数据透视表
   (a). 选中区域，点击【插入】选项卡，点击【数据透视表】按钮，弹出【创建数据透视表】对话框
   (b). 在弹出的【数据透视表字段】中，把字段拖动到相对应的【行】【列】【值】区域。
2. 值汇总方式
   (a). 点击【值】区域中的“求和项: 产量”，弹出菜单中选择【值字段设置】，弹出【值字段设置】对话框，【计算类型】进行选择
   (b). 如果对数值显示方式有要求，在弹出【值字段设置】对话框，点击左下角【数字格式】进行设置。
3. 数据透视表排序
   点击【行标签】或【列标签】右侧倒三角按钮，选择弹出菜单栏中的【其他排序选项】，在弹出的【排序】对话框，可以选择【排序选项】中的手动、升序排序还是降序排列。
4. 数据百分比计算
   (a). 再次拖动“产量”到【值】区域，“产量”字段汇总方式在【值字段设置】对话框都设置为【求和】，【值】区域内有【求和项：产量】和【求和项：产量 2】
   (b). 把光标放于【求和项：产量 2】这一列的数字上，点击鼠标右键，弹出菜单选择【值显示方式】，在右侧列表选择“列汇总百分比”，将该列表头改为“百分比”即可。
5. 从数据透视表创建数据透视图
   在表格中选择一个单元格。选择“数据透视表工具”>“分析”>“数据透视图”。选择图表。

1.21 图表

创建图表

1. 选择要用于图表的数据。

日期	班别	小组	姓名	机台	产量
2012/10/10	甲班	一组	赵明宝	1#	7849
2012/10/11	甲班	一组	吴明丽	6#	5624
2012/10/13	甲班	一组	陈星	12#	5573
2012/10/15	甲班	一组	赵大年	15#	5855
2012/10/13	乙班	一组	吴华	17#	7155
2012/10/15	乙班	二组	柳香香	13#	5588
2012/10/10	乙班	三组	张文	3#	7176
2012/10/11	乙班	三组	刘星星	16#	7045
2012/10/12	乙班	三组	朱华丽	5#	6118
2012/10/12	丙班	一组	吴小丽	8#	6277
2012/10/11	丙班	二组	陈昆	2#	5342
2012/10/11	丙班	二组	赵应芳	4#	6763
2012/10/12	丙班	二组	赵正云	7#	6867
2012/10/13	丙班	二组	王重阳	10#	6701
2012/10/10	丙班	三组	刘吴薪	9#	6840
2012/10/10	丙班	三组	吴宏艳	14#	5695
2012/10/13	丙班	三组	张大亮	11#	5491

行标签	求和项:产量	百分比
甲班	**24901**	**23.07%**
一组	24901	23.07%
乙班	**33082**	**30.64%**
一组	7155	6.63%
二组	5588	5.18%
三组	20339	18.84%
丙班	**49976**	**46.29%**
一组	6277	5.81%
二组	25673	23.78%
三组	18026	16.70%
总计	**107959**	**100.00%**

2. 单击“插入”>“推荐的图表”。
3. 在“推荐的图表”选项卡上，滚动浏览 Excel 为您的数据推荐的图表列表，然后单击任意图表以查看数据的呈现效果。
4. 如果没有看到您喜欢的图表，请单击“所有图表”以查看所有可用的图表类型。
5. 找到所要的图表时，单击该图表，然后单击“确定”。
6. 使用图表右上角附近的“图表元素”、“图表样式”和“图表筛选器”按钮添加坐标轴标题或数据标签等图表元素，自定义图表的外观或更改图表中显示的数据。
7. 若要访问其他设计和格式设置功能，请单击图表中的任意位置将“图表工具”添加到功能区，然后在“设计”和“格式”选项卡上单击所需的选项。

添加饼图

单击“插入”>“插入饼图或圆环图”，然后选择所需的图表。

饼图可以将电子表格数据的一列或一行转换到饼图中。饼图的每个扇区（数据点）显示该扇区相对于整个饼图的大小或百分比。

最适合采用饼图的情形：

- 只有一个数据系列。
- 任何数据值都不为零或小于零。
- 类别不超过七个。因为七个以上的扇区会使图表难以阅读。

除了三维饼图图表，您还可以创建复合饼图或复合条饼图。这些图表在显示一些较小的值时，将这些值分离出来放到次饼图或堆积条形图中，从而使其更易于辨别。

分离或展开饼图

- 分离单个扇区

选中要拉出的扇区，然后将该扇区从图表中心拖开。

- 分离整个饼图

右键单击饼图，然后单击“设置数据系列格式”。拖动“饼图分离”滑块以增加分隔，或在百分比框中输入数字。

在饼图中添加或删除数据

- 单击图表中的任意位置。当前显示的源数据在工作表上处于选中状态，其中显示了尺寸控点。在工作表中，拖动尺寸控点以包含新的数据或删除数据。
- 右键单击图表，然后“选择数据”。

向图表添加图例

大多数图表使用某种图例来帮助读者了解图表数据。每当在 Excel 创建图表时，都会同时自动生成图表的图例。如果图表已手动从图表中删除，则图表可能缺少图例，但可以检索丢失的图例。

单击图表。单击加号“图表元素”。选中“图例”复选框。

编辑图例文本

如果图表中的图例名称不正确，可以重命名图例项。

1. 右键单击图表。“选择数据”。
2. 在“系列”列表的图例项，然后单击“编辑”。
3. 在“系列名称”字段中，键入新的图例项。

第 2 章 PowerPoint

2.1 更改字体 16
2.1.1 更改单张幻灯片上的字体 16
2.1.2 更改整个演示文稿的字体 16
2.1.3 在 PowerPoint 中更改默认字体 16
2.1.4 更改幻灯片上页脚中的字体 17
2.2 幻灯片母版 17
2.2.1 什么是幻灯片母版？ 17
2.2.2 主题 17
2.2.3 幻灯片版式 17
2.3 动画效果 17
2.3.1 为文本和对象添加动画 17
2.3.2 选择启动动画的方式 18
2.3.3 一次显示一段文本 18
2.3.4 使文本一次显示一个字母 18
2.3.5 应用动作路径动画 18
2.3.6 在 PowerPoint 中使用平滑切换 18

2.1 更改字体

2.1.1 更改单张幻灯片上的字体

若要更改单个段落或短语的字体，请选择要更改字体的文本。

若要更改某一占位符中所有文本的字体，请选择该占位符中的所有文本，或单击该占位符。

在“开始”选项卡的“字体”组中，从“字体”列表中选择一个字体。

2.1.2 更改整个演示文稿的字体

更改整个演示文稿的字体有以下几种方法：

1. 在“设计”选项卡的“变体”组中，从“字体”列表中选择一种字体方案或者新建自定义方案。
2. 在“视图”选项卡上的“母版视图”组中，单击“幻灯片母版”。在“幻灯片母版”选项卡的“背景”组中单击“字体”，然后从列表中选择一个字体。
3. 在“开始”选项卡上的“编辑”组中，选择“替换”，然后选择“替换字体”。选择要替换的字体和要使用的字体，然后选择“替换”，再选择“关闭”。

2.1.3 在 PowerPoint 中更改默认字体

可以通过创建一个自定义模板来保存上述字体设置。此模板保存字体更新，可用于将来的演示文稿。

1. 根据自己的需要设置默认标题及正文字体并保存。
2. 点击幻灯片菜单栏【设计】左侧的【主题】下拉菜单，选择【保存当前主题】；再将右键点击该自定义主题，选择【设置为默认主题】。
3. 每次新建幻灯片，默认都是刚刚自定义的主题模版了。

2.1.4 更改幻灯片上页脚中的字体

1. 在“视图”选项卡上，单击“幻灯片母版”。
2. 在缩略图窗格的顶部，单击幻灯片母版选中它。
3. 在幻灯片母版上突出显示任何页脚元素（例如日期、页脚文本或幻灯片编号），然后在“开始”选项卡上，选择“字体”和“段落”组中所需的字体格式。

2.2 幻灯片母版

2.2.1 什么是幻灯片母版？

若要使所有的幻灯片包含相同的字体和图像（如徽标），在一个位置中便可以进行这些更改，即幻灯片母版，而这些更改将应用到所有幻灯片中。若要打开“幻灯片母版”视图，请在“视图”选项卡上选择“幻灯片母版”

母版幻灯片是窗口左侧缩略图窗格中最上方的幻灯片。与母版版式相关的幻灯片显示在此母版幻灯片下方。

编辑幻灯片母版时，基于该母版的所有幻灯片将包含这些更改。但是，所做的大部分更改最有可能成为与此母版相关的幻灯片版。

使用幻灯片母版统一演示文稿的的颜色、字体或效果。

2.2.2 主题

主题是相互辉映的一组颜色、字体和特殊效果（如阴影、反射、三维效果等）。PowerPoint 提供多种设计主题 - 包括协调的配色方案、背景、字体样式和占位符的放置。我们在“普通”视图中的“设计”选项卡上向你提供这些预先设计的主题。你还可以从 templates.office.com 获取更多主题。

您在演示文稿中使用的每个主题包括一个幻灯片母版和一组相关版式。如果希望演示文稿包含多个主题，请再添加一个幻灯片母版，并向其应用一个主题。

2.2.3 幻灯片版式

在“幻灯片母版”视图中更改和管理幻灯片版式。每个主题都有多个幻灯片版式。选择与幻灯片内容最为匹配的版式；某些版式更适用于文本内容，某些版式更适用于图形内容。

每个幻灯片版式的设置各不相同—每个版式上的不同位置中具有不同类型的占位符。

每个幻灯片母版具有名为“标题幻灯片版式”的一个相关幻灯片版式，并且每个主题针对该版式以不同方式排列文本和其他的对象占位符，具有不同的颜色、字体和效果。

你可以更改版式的任何内容以满足你的需要。当你更改版式，然后转到普通视图时，你之后添加的每张幻灯片都将基于此版式并反映版式更改后的外观。但是，如果演示文稿中有基于版式旧版本的现有幻灯片，你需要对这些幻灯片重新应用版式。

2.3 动画效果

2.3.1 为文本和对象添加动画

1. 选择要添加动画的对象或文本。
2. 选择“动画”并选择一种动画。
3. 选择“效果选项”并选择一种效果。

要对同一对象应用多个动画效果，请选择该对象，单击“添加动画”，然后选择另一个动画效果。

2.3.2 选择启动动画的方式

1. 单击时：单击幻灯片时启动动画。
2. 与上一动画同时：与序列中的上一动画同时播放动画。
3. 上一动画之后：上一动画出现后立即启动动画。
4. 持续时间：延长或缩短效果。
5. 延迟：效果运行之前增加时间。

2.3.3 一次显示一段文本

通过名为“按段落”的动画效果选项，可使列表项一次显示一段。

1. 选择包含文本的框。
2. 选择“动画”选项卡，然后选择动画。
3. 选择“效果选项”，然后选择“按段落”，使文本段落一次显示一个段落。

默认情况下，在“幻灯片放映”中演示时，每个段落都将显示以响应单击。这样，您可以控制每个段落的显示时间。可以使用功能区“动画”选项卡最右端上的“开始”、“持续时间”和“延迟”控件修改此设置。

2.3.4 使文本一次显示一个字母

您还可以通过使段落中的字符一次显示一个来创建“键入”视觉效果。

1. 在幻灯片上，选择包含文本的框。
2. 选择“动画”选项卡，然后选择动画。
3. 选择“动画窗格”，将打开“动画窗格”窗口。
4. 在“动画窗格”中，选择动画旁边的箭头，然后选择“效果选项”。
5. 在对话框中的“效果”选项卡上的“增强功能”下，选择“将文本制作为动画”旁边的箭头，然后选择“按字母”。然后，可以在“字母之间的延迟秒数”框中更改延迟时间。

2.3.5 应用动作路径动画

1. 单击要应用动作的对象。
2. 在“动画”选项卡中，单击“添加动画”。
3. 向下滚动到“动作路径”下，选择一个。
4. 如果选择“自定义路径”选项，则会绘制希望对象采用的路径。若要停止绘制自定义路径，请按 Esc。
5. 要执行某些操作（例如，更改动作路径的方向，编辑动作路径的各个点，锁定（以使其他人无法更改动画）或取消锁定动画），请单击“效果选项”。
6.

2.3.6 在 PowerPoint 中使用平滑切换

平滑切换功能可将从一张幻灯片到另一张幻灯片的平滑移动具有动画效果。

第 3 章 智慧农业系统

3.1 计算机 19
3.1.1 冯诺依曼电子计算机 19
3.2 计算机中的信息表示 20
3.2.1 整型 20
3.2.2 整型的长度 21
3.2.3 定点数 21
3.2.4 浮点数 22
3.2.5 浮点数标准 22
3.2.6 阶码为什么用无符号整数表示？ 23
3.2.7 浮点数为什么有精度损失？ 23
3.2.8 浮点数的范围和精度有多大？ 23
3.2.9 浮点数小结 23
3.3 字符编码 24
3.3.1 GB2312 24
3.3.2 GBK 24
3.3.3 UTF8 24
3.3.4 区位码、国标码、汉字机内码 24
3.3.5 汉字在计算机内部的输入和输出 25
3.4 声音、图像、视频编码 25
3.4.1 图像位深 25

3.1 计算机

3.1.1 冯诺依曼电子计算机

冯诺依曼结构的五个主要组成部分：

1. 运算器

   首先计算机要有运算处理数据的能力，所以需要一个处理单元来完成各种算数运算和逻辑运算，这就是算术逻辑单元（Arithmetic Logic Unit，ALU）。ALU 的主要功能就是在控制信号的作用下，完成加、减、乘、除等算术运算以及与、或、非、异或等逻辑运算以及移位、补位等运算。

   运算器的主要部件就是 ALU，运算器的处理对象是数据，所以数据的长度以及数据的表示方法，对运算器的影响很大。大多数通用计算机是以 16、32、64 位数据作为运算器一次处理数据的长度。能够对一个数据的所有位同时处理运算器称为并行运算器，一次只能对数据的一个位处理的运算器称为串行运算器。

   计算机运算时，运算器的操作对象和操作种类由控制器决定。运算器操作的数据从内存中读取，处理的结果再写入内存（或者暂时存放在内部寄存器中），而且运算器对内存数据的读写是由控制器来进行的。

2. 控制器

   控制器又称为控制单元（Control Unit），是计算机的神经中枢和指挥中心，只有在控制器的控制下，整个计算机才能够有条不紊地工作、自动执行程序。

   控制器的工作流程为：从内存中取指令、翻译指令、分析指令，然后根据指令的内存向有关部件发送控制命令，控制相关部件执行指令所包含的操作。

控制器和运算器共同组成中央处理器（Central Processing Unit），CPU 是一块超大规模集成电路，是计算机运算核心和控制核心，CPU 的主要功能是解释计算机指令以及处理数据。

3. 存储器

存储器的主要功能是存储程序和各种数据，并且能够在计算机运行过程高速、自动地完成程序或者数据的存储，存储器是有记忆的设备，而且采用俩种稳定状态的物理器件来记录存储信息，所以计算机中的程序和数据都要转换为二进制代码才可以存储和操作。

存储器可以分为内部存储器（内存）和外部存储器，俩者在计算机系统中各有用处，下面大概介绍一下俩种存储器的特点：

- 内部存储器

  内部存储器称为内存或者主存，是用来存放欲执行的程序和数据。

  在计算机内部，程序和数据都是以二进制代码的形式存储的，它们均以字节为单位（8 位）存储在存储器中，一个字节占用一个存储单元，并且每个存储单元都有唯一的地址号。

  CPU 可以直接使用指令对内部存储器按照地址进行读写俩种操作，读：将内存中某个存储单元的内容读出，送入 CPU 的某个寄存器中；写：在控制器的控制下，将 CPU 中某寄存器内容传到某个存储单元中。

  内存按工作方式的不同又可以分为俩部分：

  RAM：随机存储器，可以被 CPU 随机读取，一般存放 CPU 将要执行的程序、数据，断电丢失数据

  ROM：只读存储器，只能被 CPU 读，不能轻易被 CPU 写，用来存放永久性的程序和数据，比如：系统引导程序、监控程序等。具有掉电非易失性。

- 外部存储器

  外部存储器主要来存放”暂时“用不着的程序和数据，可以和内存交换数据。

  一般是磁盘、光盘、U 盘、硬盘等。

4. 输入设备
5. 输出设备

鼠标键盘是输入设备、显示器是输出设备；

手机触摸屏即时输入设备又是输出设备；

服务器中网卡既是输入设备又是输出设备；

所有的计算机程序都可以抽象为输入设备读取信息，通过 CPU 来执行存储在存储器中的程序，结果通过输出设备反馈给用户。

3.2 计算机中的信息表示

3.2.1 整型

在现代操作系统中，int 一般占用 4 个字节（Byte）的内存，共计 32 位（Bit）。如果不考虑正负数，当所有的位都为 1 时它的值最大，为 $2^{32} - 1 = 4,294,967,295 \approx 43$ 亿，这是一个很大的数，实际开发中很少用到，而诸如 1、99、12098 等较小的数使用频率反而较高。

使用 4 个字节保存较小的整数绰绰有余，会空闲出两三个字节来，这些字节就白白浪费掉了，不能再被其他数据使用。现在个人电脑的内存都比较大了，配置低的也有 2G，浪费一些内存不会带来明显的损失；而在 C 语言被发明的早期，或者在单片机和嵌入式系统中，内存都是非常稀缺的资源，所有的程序都在尽力节省内存。

反过来说，43 亿虽然已经很大，但要表示全球人口数量还是不够，必须要让整数占用更多的内存，才能表示更大的值，比如占用 6 个字节或者 8 个字节。

让整数占用更少的内存可以在 int 前边加 short，让整数占用更多的内存可以在 int 前边加 long。

3.2.2 整型的长度

我们在描述 short、int、long 类型的长度时，只对 short 使用肯定的说法，而对 int、long 使用了“一般”或者“可能”等不确定的说法。这种描述的言外之意是，只有 short 的长度是确定的，是两个字节，而 int 和 long 的长度无法确定，在不同的环境下有不同的表现。

实际情况也确实如此，C 语言并没有严格规定 short、int、long 的长度，只做了宽泛的限制：

- short 至少占用 2 个字节。
- int 建议为一个机器字长。32 位环境下机器字长为 4 字节，64 位环境下机器字长为 8 字节。
- short 的长度不能大于 int，long 的长度不能小于 int。

目前我们使用较多的 PC 系统为 Win XP、Win 7、Win 8、Win 10、Mac OS、Linux，在这些系统中，short 和 int 的长度都是固定的，分别为 2 和 4，大家可以放心使用，只有 long 的长度在 Win64 和类 Unix 系统下会有所不同，使用时要注意移植性。

类型	16 位系统/字节	32 位系统/字节	64 位系统/字节
short	2	2	2
int	2	4	4
long	4	4	8
long long	8	8	8

3.2.3 定点数

在现实生活中，我们经常使用整数和小数，不知道你有没有思考过，这些数字在计算机中是如何存储的？

如果我们想在计算机中，既能表示整数，也能表示小数，关键就在于这个小数点如何表示？

于是人们想出一种方法，即约定计算机中小数点的位置，且这个位置固定不变，小数点前、后的数字，分别用二进制表示，然后组合起来就可以把这个数字在计算机中存储起来，这种表示方式叫做「定点」表示法，用这种方法表示的数字叫做「定点数」。

定点数如果要表示整数或小数，分为以下三种情况：

1. 纯整数：例如整数 100，小数点其实在最后一位，所以忽略不写
2. 纯小数：例如：0.123，小数点固定在最高位
3. 整数 + 小数：例如 1.24、10.34，小数点在指定某个位置

以 1 个字节（8 bit）为例，我们可以约定前 5 位表示整数部分，后 3 位表示小数部分。

对于数字 1.5 用定点数表示就是这样：

$$1.5(D) = 00001100(B)$$

数字 25.125 用定点数表示就是这样：

$$25.125(D) = 11001001(B)$$

但是有没有发现一个问题，我们约定了前 5 位表示整数部分，后 3 位表示小数部分，此时这个整数部分的二进制最大值只能是 11111，即十进制的 31，小数部分的二进制最大只能表示 0.111，即十进制的 0.875。

如果我们想要表示更大范围的值，怎么办？

- 扩大 bit 的宽度：例如使用 2 个字节、4 个字节，这样整数部分和小数部分宽度增加，表示范围也就变大了
- 改变小数点的位置：小数点向后移动，整个数字范围就会扩大，但是小数部分的精度就会越来越低，没有办法表示类似 0.00001 这种高精度的值

由此我们发现，不管如何约定小数点的位置，都会存在以下问题:

- 数值的表示范围有限（小数点越靠左，整个数值范围越小）
- 数值的精度范围有限（小数点越靠右，数值精度越低）

总的来说，就是用定点数表示的小数，不仅数值的范围表示有限，而且其精度也很低。要想解决这 2 个问题，所以人们就提出了使用「浮点数」的方式表示数字。

3.2.4 浮点数

浮点数是采用科学记数法的方式来表示的，例如十进制小数 8.345，用科学记数法表示，可以有多种方式：

$$8.345 = 8.345 \times 10^0$$

$$8.345 = 83.45 \times 10^{-1}$$

$$8.345 = 834.5 \times 10^{-2}$$

看到了吗？用这种科学记数法的方式表示小数时，小数点的位置就变得「漂浮不定」了，这就是相对于定点数，浮点数名字的由来。

使用同样的规则，对于二进制数，我们也可以用科学记数法表示，也就是说把基数 10 换成 2 即可。

浮点数的格式可以写成这样：

$$V = (-1)^S \times M \times R^E$$

其中各个变量的含义如下：

- S：符号位，取值 0 或 1，决定一个数字的符号，0 表示正，1 表示负
- M：尾数，用小数表示
- R：基数，表示十进制数 R 就是 10，表示二进制数 R 就是 2
- E：指数，用整数表示

如果我们要在计算机中，用浮点数表示一个数字，只需要确认这几个变量即可。

3.2.5 浮点数标准

1985 年，IEEE 组织推出了浮点数标准，就是我们经常听到的 IEEE754 浮点数标准，这个标准统一了浮点数的表示形式，并提供了 2 种浮点格式：

- 单精度浮点数 float：32 位，符号位 S 占 1 bit，指数 E 占 8 bit，尾数 M 占 23 bit
- 双精度浮点数 double float：64 位，符号位 S 占 1 bit，指数 E 占 11 bit，尾数 M 占 52 bit

为了使其表示的数字范围、精度最大化，浮点数标准还对指数和尾数进行了规定：

- 尾数 M 的第一位总是 1（因为 $1 \le M < 2$），因此这个 1 可以省略不写，它是个隐藏位，这样单精度 23 位尾数可以表示了 24 位有效数字，双精度 52 位尾数可以表示 53 位有效数字
- 指数 E 是个无符号整数，表示 float 时，一共占 8 bit，所以它的取值范围为 0 ~ 255。但因为指数可以是负的，所以规定在存入 E 时在它原本的值加上一个中间数 127，这样 E 的取值范围为 −127 ~ 128。表示 double 时，一共占 11 bit，存入 E 时加上中间数 1023，这样取值范围为 −1023 ~ 1024。

除了规定尾数和指数位，还做了以下规定：

- 指数 E 非全 0 且非全 1：规格化数字，按上面的规则正常计算
- 指数 E 全 0：非规格化数，尾数隐藏位不再是 1，而是 0(M = 0.xxxxx)，这样可以表示 0 和很小的数
- 指数 E 全 1，尾数全 0：正无穷大/负无穷大（正负取决于 S 符号位）
- 指数 E 全 1，尾数非 0：NaN(Not a Number)。一些运算的结果不为实数或无穷时，就会返回这样的 NaN 值，比如计算 $\sqrt{-1}$ 或 $\infty - \infty$ 时。

3.2.6 阶码为什么用无符号整数表示？

如果阶码（指数）也用补码来表示，就会使得一个浮点数中出现两个符号位：浮点数自身的和浮点数指数部分的。这样的结果是，在比较两个浮点数大小时，无法像比较整数时一样使用简单的无逻辑的二进制比较。

故而浮点数的指数部分采用了移码（无符号整数）来表示。

3.2.7 浮点数为什么有精度损失？

如果我们现在想用浮点数表示 0.2，它的结果会是多少呢？

0.2 转换为二进制数的过程为，不断乘以 2，直到不存在小数为止，在这个计算过程中，得到的整数部分从上到下排列就是二进制的结果。

```
0.2 * 2 = 0.4 -> 0
0.4 * 2 = 0.8 -> 0
0.8 * 2 = 1.6 -> 1
0.6 * 2 = 1.2 -> 1
0.2 * 2 = 0.4 -> 0（发生循环）
...
```

所以 0.2(D) = 0.00110...(B)。

因为十进制的 0.2 无法精确转换成二进制小数，而计算机在表示一个数字时，宽度是有限的，无限循环的小数存储在计算机时，只能被截断，所以就会导致小数精度发生损失的情况。

3.2.8 浮点数的范围和精度有多大？

以单精度浮点数 float 为例，它能表示的最大二进制数为 $+1.111111...1 \times 2^{127}$（小数点后 23 个 1），而二进制 $1.11111...1 \approx 2$，所以 float 能表示的最大数为 $2^{128} = 3.4 \times 10^{38}$，即 float 的表示范围为：$-3.4 \times 10^{38} \sim 3.4 \times 10^{38}$。

它能表示的精度有多小呢？

float 能表示的最小二进制数为 0.0000....1（小数点后 22 个 0，1 个 1），用十进制数表示就是 $(\frac{1}{2})^{23}$。

用同样的方法可以算出，double 能表示的最大二进制数为 +1.111...111（小数点后 52 个 1）$\times 2^{1023} \approx 2^{1024} = 1.79 \times 10^{308}$，所以 double 能表示范围为：$-1.79 \times 10^{308} \sim 1.79 \times 10^{308}$。

double 的最小精度为：0.0000...1(51 个 0，1 个 1)，用十进制表示就是 $(\frac{1}{2})^{52}$。

从这里可以看出，虽然浮点数的范围和精度也有限，但其范围和精度都已非常之大，所以在计算机中，对于小数的表示我们通常会使用浮点数来存储。

3.2.9 浮点数小结

1. 浮点数一般用科学记数法表示
2. 把科学记数法中的变量，填充到固定 bit 中，即是浮点数的结果
3. 在浮点数提出的早期，各个计算机厂商各自制定自己的浮点数规则，导致不同厂商对于同一个数字的浮点数表示各不相同，在计算时还需要先进行转换才能进行计算
4. 后来 IEEE 组织提出了浮点数的标准，统一了浮点数的格式，并规定了单精度浮点数 float 和双精度浮点数 double，从此以后各个计算机厂商统一了浮点数的格式，一直延续至今
5. 浮点数在表示小数时，由于十进制小数在转换为二进制时，存在无法精确转换的情况，而在固定 bit 的计算机中存储时会被截断，所以浮点数表示小数可能存在精度损失
6. 浮点数在表示一个数字时，其范围和精度非常大，所以我们平时使用的小数，在计算机中通常用浮点数来存储

3.3 字符编码

3.3.1 GB2312

GB2312 兼容 ASCII，采用两个值 >=128 的字节表示一个汉字，也就是说，它是一个变长编码，解析原则是：

- 如果一个字节在 0 127，那么表示一个普通 ASCII 字符
- 如果一个字节在 128 255，那么继续读下一个字符（且下一个字符是 128 255），两个字符联合起来的值代表一个汉字

3.3.2 GBK

GBK 于 1995 年发布，将字符数扩充至 21919 个字符。

GBK 在编码上沿袭了 GB2312 的做法，兼容 ASCII，用双字节表示中文，但是从字符数量我们很容易看出，它不可能遵循“双字节都是 128 255”这一规范，因为这个范围只有 128，双字节能表示的则是 128×128=16384 个字符，不可能在 20000+ 的数量

所以，既要兼容 ASCII，又要限定非 ASCII 字符是双字节，那只能在第二个字节做妥协了，也就是说，GBK 中的相当多的非 ASCII 字符，是第一个字符 128 255，而第二个字符是 0 127 的，而我们知道很多文本处理是把文件当字节序列来看待，这就可能出现一些问题。

“珅”这个字在 GBK 编码中，第二个字符是竖线，干扰了日志格式

当然，双字节的第二个字节是 0 127 的，一般也都是比较生僻的汉字，但它们很多时候会出现在姓名中，在文本处理上会出现麻烦

要避免类似的问题，统一用 utf8 就好了，为什么 utf8 可以规避类似问题呢，看看 utf8 的编码规定就知道了。

3.3.3 UTF8

作为一种多字节变长编码，utf8 通过每个字节的前缀 bit 严格区分了它们的作用：

- 高位是 0 的，为单字节的 ASCII 字符
- 高位 10 的，是多字节序列中的后续字节
- 高位是 11 开头的，是多字节序列的首字节

这些规定不但使得不同字符的编码字节不会互相混淆，也确定了多字节序列的边界，抗乱码能力也非常强，丢失单字节是不会造成乱码的，很容易纠错，只有丢失了连续多个字节的时候，才可能出现错误内容。

3.3.4 区位码、国标码、汉字机内码

在网络传输的过程中：A 要给 B 传输一个数据，如果 A 传去的数据是用 GB2312-80 编的，而 B 读取地方式是用 ASCII 方式的，GB 2312-80 将区位码传过去，当 B 收到时会先读到区码，因为区码和位码的范围用十进制表示都是 0 ~ 93，如果此时的区码的十进制是 6，那么 ASCII 对应的 6 则是 ACK，会发现 ACK 是回包的指示。那么就会出现问题。

为了解决这一问题，科学家就将区位码加上 32，因为 ASCII 的通信字符是 0 ~ 32，如果将区位码的区码和位码都 +32，那么区码和位码的范围就变成了 32 ~ 125。就避开了与 ASCII 中通信字符与控制字符的相同的情况，提高了 ASCII 和区位码的兼容性。而这种 32 ~ 125 的区位码就是所谓的：国标码（GB2312 -80）。

国标码是用十六进制表示的，实际上是再区位码的区码和位码分别加上 20H(32O)，也就是加上 2020H，最后得到了 GB 2312-80。

国标码是为了防止与 ASCII 的控制字符和通信字符冲突，但是还是会和打印字符发送冲突，因此为了彻底让国标码和 ASCII 兼容。就再 GB 2312-80 的基础上分别再加上 8080H, 也就是在区码和位码上再加上 128(80H)。最

后得到了：汉字机内码，GB 2312 就是汉字机内码。

最后在计算机内部，使用的就是汉字机内码（GB 2312）

汉字机内码也有别的称呼：汉字内码、内码。

3.3.5 汉字在计算机内部的输入和输出

如果此时要用输入法打出一个“内”字，那么我需要在键盘上点 4 个键:nei1。

输入 nei1 后，输入法会将 nei1 转换成国标码，然后计算机系统或者是引用软件，会将国标码再转换成汉字机内码，最后计算机或是手机再去保存这个汉字机内码。

输入法也可能会直接将 nei1 转换成机内码。

输出：

汉字要输出，就需要用到另一种编码：汉字字形码。

汉字字形码是通过用一块一块的像素拼组而成的，1 表示这里的像素需要图上颜色，0 表示不用。从而达到输出一个汉字。

计算机会将汉字机内码转换成国标码，再将国标码转换成汉字字形码，或是直接由汉字机内码转换成汉字字形码。最后输出。

汉字机内码转换成汉字字形码需要经过以下步骤：

1. 根据汉字机内码的编码规则，将每个汉字转换成相应的二进制编码。
2. 对于一些早期的编码规则，需要使用专门的编码转换表将汉字机内码转换成 Unicode 编码。
3. 根据汉字字形库中的字形信息，将汉字机内码转换成相应的汉字字形码。
4. 在显示设备上根据汉字字形码渲染出汉字的图形形态，供用户观看和使用。

3.4 声音、图像、视频编码

3.4.1 图像位深

- 8 位：$2^8 = 2^3(R) \times 2^3(G) \times 2^2(B) = 256$ 色
- 16 位：$2^16 = 2^5(R) \times 2^6(G) \times 2^6(B) = 65536$ 色
- 24 位：$2^24 = 2^8(R) \times 2^8(G) \times 2^8(B) = 16777216$ 色
- 32 位：Alpha 透明度 + 24 位

8/16 位深度：单原色 (R/G/B) 最大只能表示为 $(0 \sim 2^3) / (0 \sim 2^6)$，显示的颜色范围有限。

24 位：每个像素点由 8bit 红 8bit 绿 8bit 蓝组合颜色而成，每一个原始颜色 (R/G/B) 都可以完全显示 (0~0xff)，所以 24 位及以上，我们就叫做真彩色。

RGB 色彩系统可根据 Red，Green 和 Blue 颜色的组合来构造所有颜色。

红色，绿色和蓝色分别使用 8 位，其整数值介于 0 到 255 之间。这使 256 * 256 * 256 = 16777216 种可能的颜色成为可能。

LED 监视器中的每个像素通过红色，绿色和蓝色 LED（发光二极管）的组合以这种方式显示颜色。

当红色像素设置为 0 时，LED 熄灭。当红色像素设置为 255 时，LED 完全打开。

它们之间的任何值会将 LED 设置为部分发光。

第 4 章 农业数字化技术

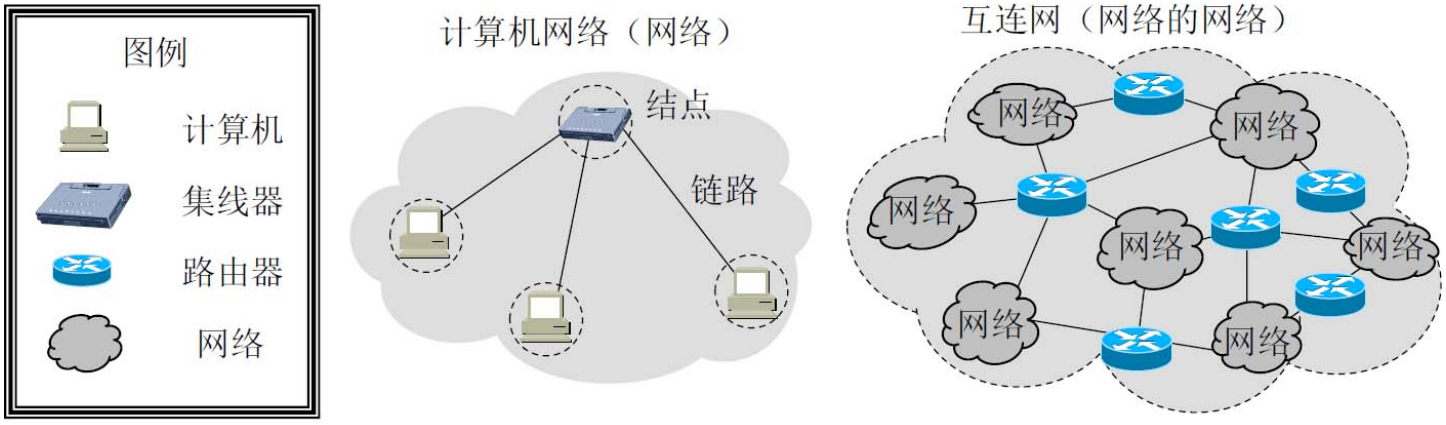
第 5 章 数据传输技术

5.1 互联网 27
5.1.1 什么是互联网 27
5.1.2 互联网基础结构发展的三个阶段 28
5.1.3 互联网 + ... 28
5.2 网址是什么？ 29
5.3 IP 地址 30
5.4 DNS 域名系统 32
5.5 互联网的三种文档 33
5.6 计算机网络体系结构 34
5.6.1 计算机网络体系结构的形成 34
5.6.2 计算机网络的三种体系结构 35
5.6.3 各层的作用 36
5.7 协议端口 37
5.8 TCP/IP 多路复用 38
5.9 三次握手 38
5.10 四次挥手 39
5.11 TCP 可靠通信的具体实现 39
5.12 TCP 报文段首部格式 40
5.13 UDP 的首部格式 41
5.14 ARP 地址解析协议 42
5.15 链路层 42
5.16 虚拟专用网 VPN 43

5.1 互联网

5.1.1 什么是互联网

计算机网络（简称为网络）由若干结点（node）和连接这些结点的链路（link）组成。网络中的结点可以是计算机、集线器、交换机或路由器等。网络之间还可以通过路由器互连起来，这就构成了一个覆盖范围更大的计算机网络。这样的网络称为互连网（internetwork 或 internet）。因此互连网是“网络的网络”（network of networks）。



网络把许多计算机连接在一起，而互连网则把许多网络通过路由器连接在一起。与网络相连的计算机常称为主机。还有一点也必须注意，就是网络互连并不是把计算机仅仅简单地在物理上连接起来，因为这样做并不能达到计算机之间能够相互交换信息的目的。我们还必须在计算机上安装许多使计算机能够交换信息的软件才行。

5.1.2 互联网基础结构发展的三个阶段

1. 从单个网络 ARPANET 向互连网发展

   1969 年美国国防部创建的第一个分组交换网 ARPANET 最初只是一个单个的分组交换网（并不是一个互连的网络）。所有要连接在 ARPANET 上的主机都直接与就近的结点交换机相连。但到了 20 世纪 70 年代中期，人们已认识到不可能仅使用一个单独的网络来满足所有的通信问题。于是 ARPA 开始研究多种网络（如分组无线电网络）互连的技术，这就导致了互连网络的出现。这就成为现今互联网（Internet）的雏形。1983 年 TCP/IP 协议成为 ARPANET 上的标准协议，使得所有使用 TCP/IP 协议的计算机都能利用互连网相互通信，因而人们就把 1983 年作为互联网的诞生时间。1990 年 ARPANET 正式宣布关闭，因为它的实验任务已经完成。

2. 三级结构的互联网

   从 1985 年起，美国国家科学基金会 NSF（NationalScienceFoundation）就围绕六个大型计算机中心建设计算机网络，即国家科学基金网 NSFNET。它是一个三级计算机网络，分为主干网、地区网和校园网（或企业网）。这种三级计算机网络覆盖了全美国主要的大学和研究所，并且成为互联网中的主要组成部分。1991 年，NSF 和美国的其他政府机构开始认识到，互联网必将扩大其使用范围，不应仅限于大学和研究机构。世界上的许多公司纷纷接入到互联网，网络上的通信量急剧增大，使互联网的容量已满足不了需要。于是美国政府决定将互联网的主干网转交给私人公司来经营，并开始对接入互联网的单位收费。

3. 多层次 ISP 结构的互联网

   从 1993 年开始，由美国政府资助的 NSFNET 逐渐被若干个商用的互联网主干网替代，而政府机构不再负责互联网的运营。这样就出现了一个新的名词：互联网服务提供者 ISP（InternetServiceProvider）。在许多情况下，ISP 就是一个进行商业活动的公司，因此 ISP 又常译为互联网服务提供商。例如，中国电信、中国联通和中国移动等公司都是我国最有名的 ISP。

   ISP 可以从互联网管理机构申请到很多 IP 地址（互联网上的主机都必须有 IP 地址才能上网，这一概念我们将在第 4 章的 4.2.2 节详细讨论），同时拥有通信线路（大 ISP 自己建造通信线路，小 ISP 则向电信公司租用通信线路）以及路由器等连网设备，因此任何机构和个人只要向某个 ISP 交纳规定的费用，就可从该 ISP 获取所需 IP 地址的使用权，并可通过该 ISP 接入到互联网。IP 地址的管理机构不会把一个单个的 IP 地址分配给单个用户（不“零售”IP 地址），而是把一批 IP 地址有偿租赁给经审查合格的 ISP（只“批发”IP 地址）。由此可见，现在的互联网已不是某个单个组织所拥有而是全世界无数大大小小的 ISP 所共同拥有的，这就是互联网也称为“网络的网络”的原因。

5.1.3 互联网 +

现在常常可以看到一种新的提法，即“互联网 +”。它的意思就是“互联网 + 各个传统行业”，因此可以利用信息通信技术和互联网平台来创造新的发展生态。实际上“互联网 +”代表一种新的经济形态，其特点就是把互联网的创新成果深度融合于经济社会各领域之中，这就大大地提升了实体经济的创新力和生产力。我们也必须看到互联网的各种应用对各行各业的巨大冲击。例如，电子邮件迫使传统的电报业务退出市场。网络电话的普及使得传统的长途电话（尤其是国际长途电话）的通信量急剧下降。对日用商品快捷方便的网购造成了不少实体商店的停业。原来必须排长队购买火车票的网点已被非常方便的网购所替代。网约车的问世对出租车行业产生了的巨大冲击。这些例子说明了互联网应用已对整个社会的各领域产生了很大的影响。

中国国际“互联网 +”大学生创新创业大赛是由教育部与政府、各高校共同主办的赛事。大赛要求参赛项目能够将移动互联网、云计算、大数据、人工智能、物联网、下一代通讯技术、区块链等新一代信息技术与经济社会各

领域紧密结合，服务新型基础设施建设，培育新产品、新服务、新业态、新模式；发挥互联网在促进产业升级以及信息化和工业化深度融合中的作用，促进制造业、农业、能源、环保等产业转型升级；发挥互联网在社会服务中的作用，创新网络化服务模式，促进互联网与教育、医疗、交通、金融、消费生活等深度融合。

要从实际问题中来，到实际问题中去，来解决实际问题，项目创新点是一个项目的核心，能达到的最终高度关键取决于核心思路。

5.2 网址是什么？

网址（URL）的构成

URL : Uniform Resource Locator，统一资源定位符

https://www.hebau.edu.cn/info/1062/15713.htm

- 协议：http
- 服务：www (万维网)
- 域名：hebau.edu.cn
- 网站类型：edu.cn
- 目录：info/1062
- 文件：15713.htm

常见网址协议 (scheme)

- http

  超文本传输协议（HyperText Transfer Protocol）

  互联网应用最广泛的一种网络协议，客户端和服务器端请求和答应的标准。
- https

  由于 HTTP 天生“明文”的特点，整个传输过程完全透明，任何人都能够在链路中截获、修改或者伪造请求/响应报文，数据不具有可信性。因此就诞生了为安全而生的 HTTPS 协议。这是加强版 http 协议，是一种更加安全的数据传输协议。
- ftp

  访问远程 FTP 服务器上的文件。
- file

  访问本地磁盘上的文件。
- mailto

  邮件地址协议

  HTTP 和 HTTPS 的协议名称后面，紧跟着一个冒号和两个斜杠（://）。其他协议不一定如此，邮件地址协议 mailto: 的协议名后面只有一个冒号，比如 mailto:foo@example.com。

端口

端口紧跟在域名后面，两者之间使用冒号分隔。

比如 www.example.com:80。

表 5.1: 默认端口

http	80
https	443
ftp	21
socks	1080

网站类型

.com 国际最广泛流行的通用域名格式

.cn 中国国家域名

.org 非赢利性组织

.edu 教育机构

.gov 国家政府机构

路径

路径（path）是资源在网站的位置。比如，/info/1062/15713.htm 这个路径，指向网站的/info/1062 子目录下面的网页文件 15713.htm。

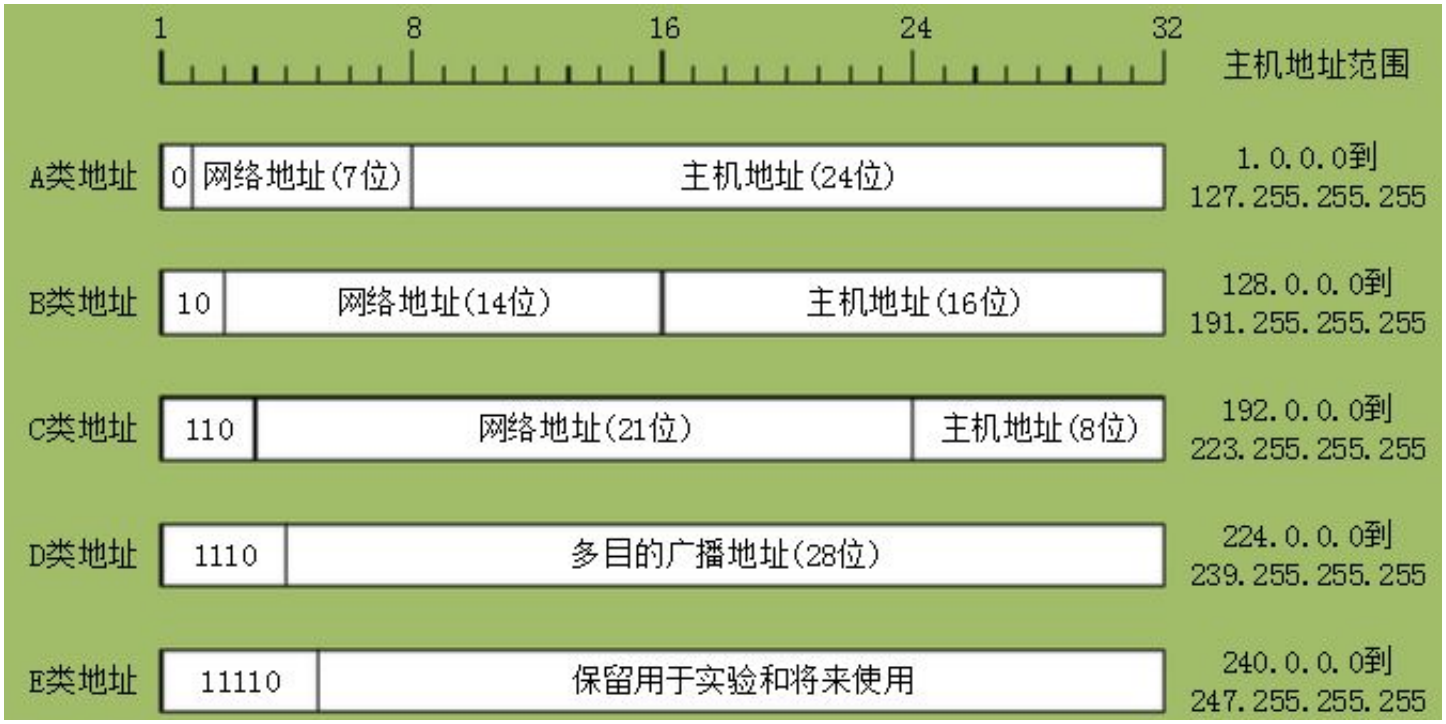
互联网的早期，路径是真实存在的物理位置。现在由于服务器可以模拟这些位置，所以路径只是虚拟位置。

路径可能只包含目录，不包含文件名，比如/foo/，甚至结尾的斜杠都可以省略。这时，服务器通常会默认跳转到该目录里面的 index.html 文件（即等同于请求/foo/index.html），但也可能有其他的处理（比如列出目录里面的所有文件），这取决于服务器的设置。一般来说，访问 www.example.com 这个网址，很可能返回的是网页文件 www.example.com/index.html。

5.3 IP 地址

IP 地址的格式

IP 地址 = 网络地址 + 主机地址



最初设计互联网络时，为了便于寻址以及层次化构造网络，每个 IP 地址包括两个标识码（ID），即网络 ID 和主机 ID。同一个物理网络上的所有主机都使用同一个网络 ID，网络上的一个主机（包括网络上工作站，服务器和路由器等）有一个主机 ID 与其对应。IP 地址根据网络 ID 的不同分为 5 种类型，A 类地址、B 类地址、C 类地址、D 类地址和 E 类地址。

IP 地址的分类

1. A 类 IP 地址

   个 A 类 IP 地址由 1 字节的网络地址和 3 字节主机地址组成，网络地址的最高位必须是“0”，地址范围从 0.0.0.0～126.255.255.255，默认子网掩码为：255.0.0.0。可用的 A 类网络有 126 个，每个网络能容纳 1677 多

万个主机。A 类地址分配给规模特别大的网络使用，例如 IBM 公司的网络。A 类 IP 地址占 IP 地址总数比例 1/2。

2. B 类 IP 地址

一个 B 类 IP 地址由 2 个字节的网络地址和 2 个字节的主机地址组成，网络地址的最高位必须是“10”，地址范围从 128.0.0.0 到 191.255.255.255，默认子网掩码为：255.255.0.0。可用的 B 类网络有 16382 个，每个网络能容纳 6 万多个主机。B 类 IP 地址占 IP 地址总数比例 1/4。B 类地址分配给一般的中型网络。

3. C 类 IP 地址

一个 C 类 IP 地址由 3 字节的网络地址和 1 字节的主机地址组成，网络地址的最高位必须是“110”。范围从 192.0.0.0 到 223.255.255.255，默认子网掩码为：255.255.255.0。C 类网络可达 209 万余个，每个网络能容纳 254 个主机。C 类 IP 地址占 IP 地址总数比例 1/8。C 类地址分配给小型网络，如一般的局域网和校园网，它可连接的主机数量是最少的，采用把所属的用户分为若干的网段进行管理。

4. D 类 IP 地址

用于多点广播（Multicast）。D 类 IP 地址第一个字节以“1110”开始，它是一个专门保留的地址。

5. E 类 IP 地址

以“11110”开始，为将来使用保留。

IP 私有地址

私有 IP 就是在本地局域网上的 IP，与之对应的是公有 IP（在互联网上的 IP）。

随着私有 IP 网络的发展，为节省可分配的注册 IP 地址，有一组 IP 地址被拿出来专门用于私有 IP 网络，称为私有 IP 地址。

在 IP 地址 3 种主要类型里，各保留了 3 个区域作为私有地址，其地址范围如下：

A 类地址：10.0.0.0～10.255.255.255

B 类地址：172.16.0.0～172.31.255.255

C 类地址：192.168.0.0～192.168.255.255

判断两个 IP 地址是否是同一个网段中

要判断两个 IP 地址是不是在同一个网段，就将它们的 IP 地址分别与子网掩码做与运算，得到的结果一网络号，如果网络号相同，就在同一子网，否则，不在同一子网。

例：假定选择了子网掩码 255.255.254.0，现在分别将上述两个 IP 地址分别与掩码做与运算：

```
211.95.165.24 11010011 01011111 10100101 00011000
255.255.254.0 11111111 11111111 111111110 00000000
与的结果是: 11010011 01011111 10100100 00000000
211.95.164.78 11010011 01011111 10100100 01001110
255.255.254.0 11111111 11111111 111111110 00000000
与的结果是: 11010011 01011111 10100100 00000000
```

可以看出, 得到的结果 (这个结果就是网络地址) 都是一样的，因此可以判断这两个 IP 地址在同一个子网。

子网掩码

子网掩码是一个 32 位的 2 进制数，其对应网络地址的所有位都置为 1，对应于主机地址的所有位置都为 0。子网掩码告知路由器，地址的哪一部分是网络地址，哪一部分是主机地址，使路由器正确判断任意 IP 地址是否是本网段的，从而正确地进行路由。

网掩码不能单独存在，它必须结合 IP 地址一起使用。子网掩码的主要作用有两个，一是用于屏蔽 IP 地址的一部分以区别网络标识和主机标识，并说明该 IP 地址是在局域网上，还是在远程网上。二是用于将一个大的 IP 网络划分为若干小的子网络。

5.4 DNS 域名系统

一个疑问

首先我们要清楚以下几点：

- 互联网上的所有数据都是存储在主机 (服务器) 上
- 互联网中的所有主机都拥有唯一的 IP 地址
- 互联网中任意两台主机通信都是通过 IP 地址来实现

> 如何查看网站 ip 地址：
> cmd
> ping www.baidu.com

我们以两台主机最简单的通信方式——上网为例，我们上网的实质就是获取网址对应主机上的数据并在用户主机上进行展示 (浏览器上)，那么我们就该怀疑一个问题：互联网中的任意两台主机通信是依靠 IP 地址进行的，而我们上网只是输入的网址，并不是 IP 地址，怎么就能找到对方主机并获取它的数据呢?

域名系统

域名系统 (Domain Name System, DNS) 是因特网使用的命名系统，用来把便于人们记忆的具有特定含义的主机名 (如 www.example.com) 转换为便于机器处理的 IP 地址。

相对于 IP 地址，人们更喜欢使用具有特定含义的字符串来标识因特网上的计算机。

因特网采用层次树状结构的命名方法。采用这种命名方法，任何一个连接到因特网的主机或路由器，都有一个唯一的层次结构名称，即域名 (Domain Name)。

域还可以划分为子域，而子域还可以继续划分为子域的子域，这样就形成了顶级域、二级域、三级域等。

域名服务器

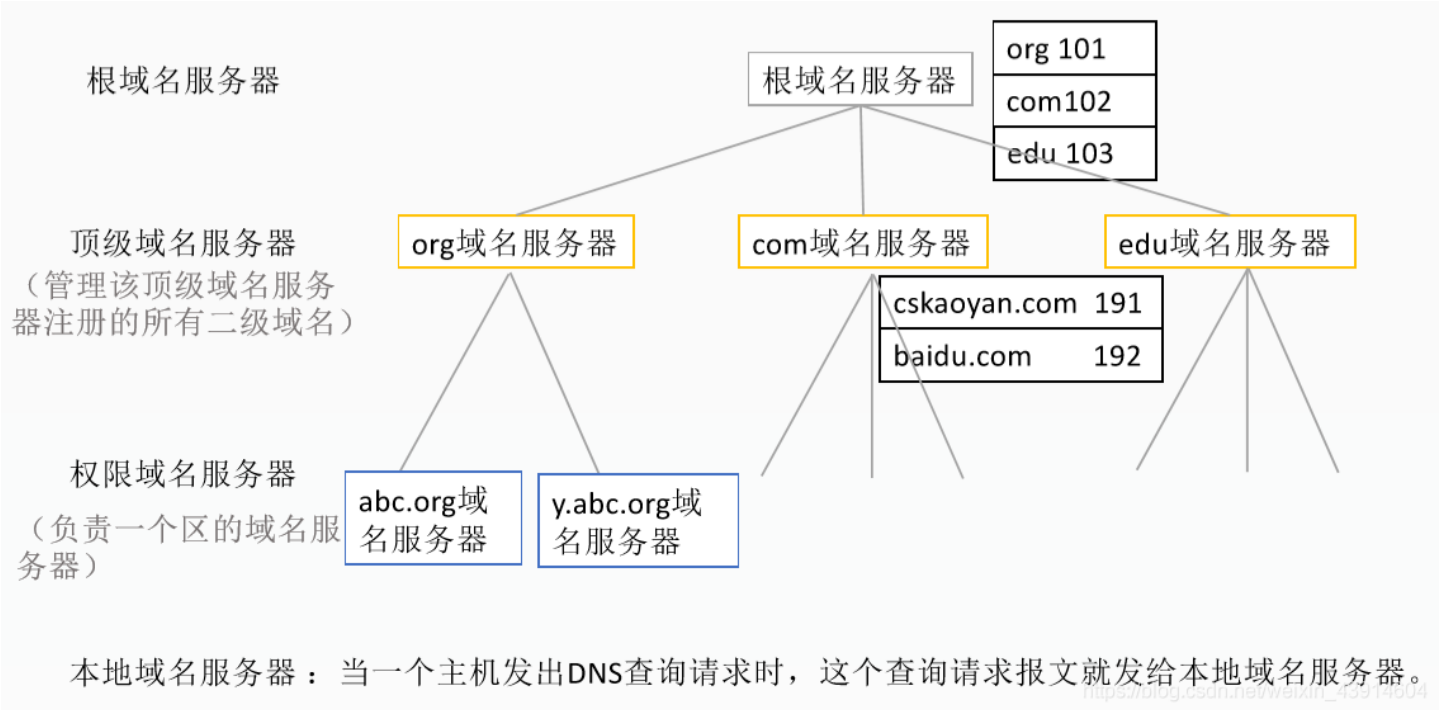
因特网的域名系统被设计成一个联机分布式的数据库系统，并采用客户/服务器模型。

域名到 IP 地址的解析是由运行在域名服务器上的程序完成的。

每个域名服务器不但能够进行一些域名到 IP 地址的解析，而且还必须具有连向其他域名服务器的信息。当自己不能进行域名到 IP 地址的转换时，能够知道到什么地方去找其他域名服务器。

DNS 使用了大量的域名服务器，它们以层次方式组织。没有一台域名服务器具有因特网上所有主机的映射，相反，该映射分布在所有的 DNS 上。

采用分布式设计的 DNS，是一个在因特网上实现分布式数据库的精彩范例。主要有 4 种类型的域名服务器。



- 根域名服务器

  根域名服务器是最高层次的域名服务器，所有的根域名服务器都知道所有的顶级域名服务器的 IP 地址。

  根域名服务器也是最重要的域名服务器，不管是哪个本地域名服务器，若要对因特网上任何一个域名进行解析，只要自己无法解析，就首先要求助于根域名服务器。

  根域名服务器用来管辖顶级域 (如.com)，通常它并不直接把待查询的域名直接转换成 IP 地址，而是告诉本地域名服务器下一步应当找哪个顶级域名服务器进行查询。
- 顶级域名服务器

  这些域名服务器负责管理在该顶级域名服务器注册的所有二级域名。

  收到 DNS 查询请求时, 就给出相应的回答
- 权限域名服务器

  每台主机都必须在授权域名服务器处登记。为了更加可靠地工作，一台主机最好至少有两个授权域名服务器。

  实际上，许多域名服务器都同时充当本地域名服务器和授权域名服务器。

  授权域名服务器总能将其管辖的主机名转换为该主机的 IP 地址。
- 本地域名服务器

  本地域名服务器对域名系统非常重要。

  每个因特网服务提供者 (ISP)，或一所大学，甚至一所大学中的各个系，都可以拥有一个本地域名服务器。

  当一台主机发出 DNS 查询请求时，这个查询请求报文就发送给该主机的本地域名服务器。

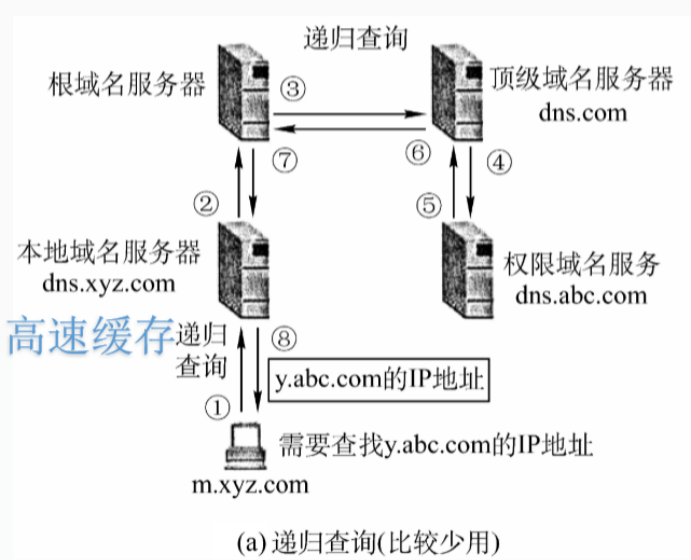
  事实上，我们在 Windows 系统中配置“本地连接”时，就需要填写 DNS 地址，这个地址就是本地 DNS (域名服务器) 的地址。

递归查询，迭代查询

5.5 互联网的三种文档

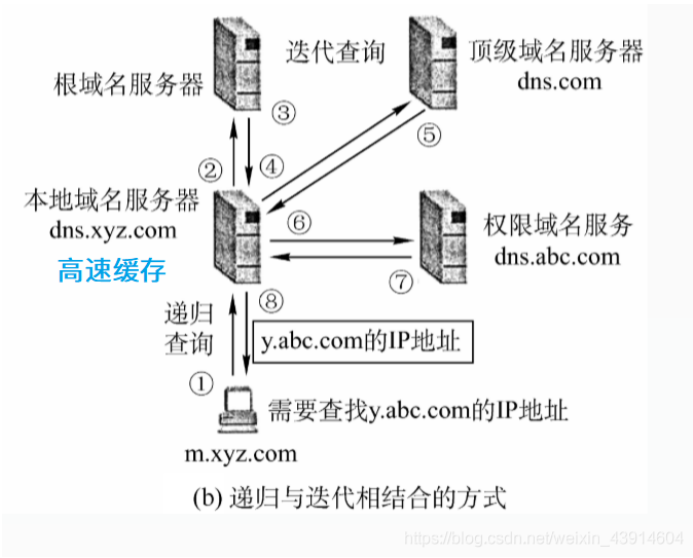
1. 静态文档

静态文档是指内容固定的文档，它是由万维网服务器创建，并存放在其中。



(a) 递归查询(比较少用)

https://blog.csdn.net/weixin_43914604



(b) 递归与迭代相结合的方式

https://blog.csdn.net/weixin_43914604

当客户利用浏览器访问万维网服务器里的该文档时，这个文档的副本被传送到客户，客户就可使用浏览程序显示这个文档。当然，服务器中的文档内容是可以修改的，但客户却不能修改它。

静态文档的最大优点是简单，文档可以由非程序设计人员来创建。它的缺点是不够灵活。因此，对于内容变化频繁的文档是不适合做成静态文档的。

2. 动态文档

所谓“动态”，并不是指网页的上动态效果，例如：视频、动图以及滚动字幕等。

所谓的动态网页，是指跟静态网页相对的一种网页编程技术。静态网页，随着 html 代码的生成，页面的内容和显示效果就基本上不会发生变化了——除非你修改页面代码。动态网页则不然，页面代码虽然没有变，但是显示的内容却是可以随着时间、环境或者数据库操作的结果而发生改变的。

动态文档是指文档的内容是在浏览器访问服务器时才得以创建。当浏览器的请求到达时，服务器就运行一个创建动态文档的应用程序。

动态文档的最大优点是具有告知当前最新信息的能力。

该应用程序对调览器发送来的数据进行处理，服务器把该程序或脚本的输出作为对浏览器请求该文档的响应。由于浏览器每次请求的响应都是动态生成的，因此每一个请求所得到的动态文档的内容也不一样。

3. 活动文档

活动文档是指能够提供了一种连续更新屏幕内容的技术，这种技术把创建文档的工作移到浏览器端进行。

当浏览器请求一个活动文档时，服务器就返回这个活动文档程序的副本或脚本，然后就在浏览器端运行，此时，活动文档程序可与用户直接交互，以便连续地更新屏幕的显示内容。

> 静态文档和动态文档都是纯 HTML 文档，但是其产生的机制和静态万维网文档是有区别。
> 活动文档是 HTML 的文档添加了其他的一些编程

5.6 计算机网络体系结构

5.6.1 计算机网络体系结构的形成

计算机网络体系结构的形成计算机网络是个非常复杂的系统。为了说明这一点，可以设想一种最简单的情况：连接在网络上的两台计算机要互相传送文件。显然，在这两台计算机之间必须有一条传送数据的通路。但这还远远不够。至少还有以下几项工作需要去完成：

1. 发起通信的计算机必须将数据通信的通路进行激活（activate）。所谓“激活”就是要发出一些信令，保证要传送的计算机数据能在这条通路上正确发送和接收。
2. 要告诉网络如何识别接收数据的计算机。
3. 发起通信的计算机必须查明对方计算机是否已开机，并且与网络连接正常。
4. 发起通信的计算机中的应用程序必须弄清楚，在对方计算机中的文件管理程序是否已做好接收文件和存储文件的准备工作。
5. 若计算机的文件格式不兼容，则至少其中一台计算机应完成格式转换功能。
6. 对出现的各种差错和意外事故，如数据传送错误、重复或丢失，网络中某个结点交换机出现故障等，应当有可靠的措施保证对方计算机最终能够收到正确的文件。

由此可见，相互通信的两个计算机系统必须高度协调工作才行，而这种“协调”是相当复杂的。为了设计这样复杂的计算机网络，早在最初的 ARPANET 设计时即提出了分层的方法。“分层”可将庞大而复杂的问题，转化为若干较小的局部问题，而这些较小的局部问题就比较易于研究和处理。

1974 年，美国的 IBM 公司宣布了系统网络体系结构 SNA（System Network Architecture）。这个著名的网络标准就是按照分层的方法制定的。现在用 IBM 大型机构建的专用网络仍在使用 SNA。不久后，其他一些公司也相继推出自己公司的具有不同名称的体系结构。

不同的网络体系结构出现后，使用同一个公司生产的各种设备都能够很容易地互连成网。这种情况显然有利于一个公司垄断市场。但由于网络体系结构的不同，不同公司的设备很难互相连通。

然而，全球经济的发展使得不同网络体系结构的用户迫切要求能够互相交换信息。为了使不同体系结构的计算机网络都能互连，国际标准化组织 ISO 于 1977 年成立了专门机构研究该问题。他们提出了一个试图使各种计算机在世界范围内互连成网的标准框架，即著名的开放系统互连基本参考模型 OSI/RM（Open Systems Interconnection Reference Model），简称为 OSI，也就是所谓的七层协议的体系结构。

OSI 试图达到一种理想境界，即全球计算机网络都遵循这个统一标准，因而全球的计算机将能够很方便地进行互连和交换数据。在 20 世纪 80 年代，许多大公司甚至一些国家的政府机构纷纷表示支持 OSI。当时看来似乎在不久的将来全世界一定会按照 OSI 制定的标准来构造自己的计算机网络。然而到了 20 世纪 90 年代初期，虽然整套的 OSI 国际标准都已经制定出来了，但由于基于 TCP/IP 的互联网已抢先在全球相当大的范围成功地运行了，而与此同时却几乎找不到有什么厂家生产出符合 OSI 标准的商用产品。因此人们得出这样的结论：OSI 只获得了一些理论研究的成果，但在市场化方面则事与愿违地失败了。现今规模最大的、覆盖全球的、基于 TCP/IP 的互联网并未使用 OSI 标准。

OSI 失败的原因可归纳为：

1. OSI 的专家们缺乏实际经验，他们在完成 OSI 标准时缺乏商业驱动力；
2. OSI 的协议实现起来过分复杂，而且运行效率很低；
3. OSI 标准的制定周期太长，因而使得按 OSI 标准生产的设备无法及时进入市场；
4. OSI 的层次划分不太合理，有些功能在多个层次中重复出现。

按照一般的概念，网络技术和设备只有符合有关的国际标准才能大范围地获得工程上的应用。但现在情况却反过来了。得到最广泛应用的不是法律上的国际标准 OSI，而是非国际标准 TCP/IP。这样，TCP/IP 就常被称为是事实上的国际标准。从这种意义上说，能够占领市场的就是标准。在过去制定标准的组织中往往以专家、学者为主。但现在许多公司都纷纷加入各种标准化组织，使得技术标准具有浓厚的商业气息。一个新标准的出现，有时不一定反映其技术水平是最先进的，而是往往有着一定的市场背景。

5.6.2 计算机网络的三种体系结构

1. OSI 七层协议体系结构

OSI 是由国际标准化组织制定的标准，它概念清楚，理论完善，但是复杂又不实用。

2. TCP/IP 的四层协议

TCP/IP 体系结构得到了非常广泛的应用。它包含应用层、运输层、网际层和网络接口层（用网际层这个名字是强调这一层是为了解决不同网络的互连问题）。不过从实质上讲，TCP/IP 只有最上面的三层，因为最下面的网络接口层并没有什么具体内容。

3. 五层协议的体系结构

学习计算机网络时我们一般采用折中的办法，也就是中和 OSI 和 TCP/IP 的优点，采用一种只有五层协议的体系结构，这样既简洁又能将概念阐述清楚。

5.6.3 各层的作用

1. 应用层

应用层 (application-layer) 的任务是通过应用进程间的交互来完成特定网络应用。应用层协议定义的是应用进程（进程：主机中正在运行的程序）间的通信和交互的规则。对于不同的网络应用需要不同的应用层协议。在互联网中应用层协议很多，如域名系统 DNS，支持万维网应用的 HTTP 协议，支持电子邮件的 SMTP 协议等等。我们把应用层交互的数据单元称为报文。

2. 运输层

运输层 (transport layer) 的主要任务就是负责向两台主机进程之间的通信提供通用的数据传输服务。应用进程利用该服务传送应用层报文。“通用的”是指并不针对某一个特定的网络应用，而是多种应用可以使用同一个运输层服务。由于一台主机可同时运行多个线程，因此运输层有复用和分用的功能。所谓复用就是指多个应用层进程可同时使用下面运输层的服务，分用和复用相反，是运输层把收到的信息分别交付上面应用层中的相应进程。

运输层主要使用以下两种协议

传输控制协议 TCP（Transmisson Control Protocol）–提供面向连接的，可靠的数据传输服务。

用户数据协议 UDP（User Datagram Protocol）–提供无连接的，尽最大努力的数据传输服务（不保证数据传输的可靠性）。

TCP 的主要特点

1. TCP 是面向连接的。（就好像打电话一样，通话前需要先拨号建立连接，通话结束后要挂机释放连接）；

2. 每一条 TCP 连接只能有两个端点，每一条 TCP 连接只能是点对点的（一对一）；

3. TCP 提供可靠交付的服务。通过 TCP 连接传送的数据，无差错、不丢失、不重复、并且按序到达；

4. TCP 提供全双工通信。TCP 允许通信双方的应用进程在任何时候都能发送数据。TCP 连接的两端都设有发送缓存和接收缓存，用来临时存放双方通信的数据；

5. 面向字节流。TCP 中的“流”（Stream）指的是流入进程或从进程流出的字节序列。“面向字节流”的含义是：虽然应用程序和 TCP 的交互是一次一个数据块（大小不等），但 TCP 把应用程序交下来的数据仅仅看成是一连串的无结构的字节流。

3. 网络层

在计算机网络中进行通信的两个计算机之间可能会经过很多个数据链路，也可能还要经过很多通信子网。网络层的任务就是选择合适的网间路由和交换结点，确保数据及时传送。在发送数据时，网络层把运输层产生的报文段或用户数据报封装成分组和包进行传送。在 TCP/IP 体系结构中，由于网络层使用 IP 协议，因此分组也叫 IP 数据报，简称数据报。

网络层中的“网络”二字已经不是我们通常谈到的具体网络，而是指计算机网络体系结构模型中第三层的名称.

互联网是由大量的异构（heterogeneous）网络通过路由器（router）相互连接起来的。互联网使用的网络层协

议是无连接的网际协议 IP（Internet Protocol）和许多种路由选择协议，因此互联网的网络层也叫做网际层或 IP 层。网络层、网际层和 IP 层都是同义语。

4. 数据链路层

数据链路层 (data link layer) 通常简称为链路层。两台主机之间的数据传输，总是在一段一段的链路上传送的，这就需要使用专门的链路层的协议。在两个相邻节点之间传送数据时，数据链路层将网络层交下来的 IP 数据报组装程帧，在两个相邻节点间的链路上传送帧。每一帧包括数据和必要的控制信息（如同步信息，地址信息，差错控制等）。

在接收数据时，控制信息使接收端能够知道一个帧从哪个比特开始和到哪个比特结束。这样，数据链路层在收到一个帧后，就可从中提出数据部分，上交给网络层。控制信息还使接收端能够检测到所收到的帧中有误差错。如果发现差错，数据链路层就简单地丢弃这个出了差错的帧，以避免继续在网络中传送下去白白浪费网络资源。如果需要改正数据在链路层传输时出现差错（这就是说，数据链路层不仅要检错，而且还要纠错），那么就要采用可靠性传输协议来纠正出现的差错。这种方法会使链路层的协议复杂些。

5. 物理层

在物理层上所传送的数据单位是比特。物理层 (physical layer) 的作用是实现相邻计算机节点之间比特流的透明传送，尽可能屏蔽掉具体传输介质和物理设备的差异。使其上面的数据链路层不必考虑网络的具体传输介质是什么。“透明传送比特流”表示经实际电路传送后的比特流没有发生变化，对传送的比特流来说，这个电路好像是看不见的。

在互联网使用的各种协中最重要和最著名的就是 TCP/IP 两个协议。现在人们经常提到的 TCP/IP 并不一定单指 TCP 和 IP 这两个具体的协议，而往往表示互联网所使用的整个 TCP/IP 协议族。

5.7 协议端口

在网络技术中，端口（Port）有好几种意思。集线器、交换机、路由器的端口指的是连接其他网络设备的接口，如 RJ-45 端口、Serial 端口等。我们这里所指的端口不是指物理意义上的端口，而是特指 TCP/IP 协议中的端口，是逻辑意义上的端口。

如果把 IP 地址比作一间房子，端口就是出入这间房子的门。真正的房子只有几个门，但是一个 IP 地址的端口可以有 65536（即：2^{16}）个之多！端口是通过端口号来标记的，端口号只有整数，范围是从 0 到 65535（$2^{16}-1$）。

在 Internet 上，各主机间通过 TCP/IP 协议发送和接收数据包，各个数据包根据其目的主机的 ip 地址来进行互联网络中的路由选择, 把数据包顺利的传送到目的主机。大多数操作系统都支持多程序（进程）同时运行，那么目的主机应该把接收到的数据包传送给众多同时运行的进程中的哪一个呢？显然这个问题有待解决，端口机制便由此被引入进来。

本地操作系统会给那些有需求的进程分配协议端口（protocol port，即我们常说的端口），每个协议端口由一个正整数标识，如：80，139，445，等等。当目的主机接收到数据包后，将根据报文首部的目的端口号，把数据发送到相应端口，而与此端口相对应的那个进程将会领取数据并等待下一组数据的到来。

TCP 连接状态详解

LISTEN：侦听来自远方的 TCP 端口的连接请求

SYN-SENT：再发送连接请求后等待匹配的连接请求

SYN-RECEIVED：再收到和发送一个连接请求后等待对方对连接请求的确认

ESTABLISHED：代表一个打开的连接

FIN-WAIT-1：等待远程 TCP 连接中断请求，或先前的连接中断请求的确认

FIN-WAIT-2：从远程 TCP 等待连接中断请求

CLOSE-WAIT：等待从本地用户发来的连接中断请求
CLOSING：等待远程 TCP 对连接中断的确认
LAST-ACK：等待原来的发向远程 TCP 的连接中断请求的确认
TIME-WAIT：等待足够的时间以确保远程 TCP 接收到连接中断请求的确认
CLOSED：没有任何连接状态

查看端口命令　netstat (选项)
-a 或-all 列出所有端口 (包含 TCP 和 UDP)；
-at 列出所有 TCP 端口
-au 列出所有 UDP 端口
-l 列出所有处于监听状态的端口
-o 显示拥有的与每个连接关联的进程 PID

5.8 TCP/IP 多路复用

所有网络通信的本质目标就是进程间通信。

除了寻址（Addressing），IP 协议还有一个非常重要的能力就是路由。寻址告诉我们去往下一个目的地该朝哪个方向走，路由则是根据下一个目的地选择路径。寻址更像在导航，路由更像在操作方向盘。

多路复用

一台机器上的应用可以有很多。但是实际的出口设备，比如说网卡、网线通常只有一份。因此这里需要用到一个叫作多路复用（Multiplex）的技术。多路复用，就是多个信号，复用一个信道。

传输层多路复用

多个请求复用一个 TCP 连接。

多个请求相当于并行的发送请求。即使其中某个请求发生故障，也不会阻塞其他请求。从这个角度看，多路复用实际上是一种 Non-Blocking（非阻塞）的技术。

如果其中一个数据段（TCP Segment）发生异常，只影响其中一个颜色的请求，其他请求仍然可以到达服务。

网络层多路复用

两个应用之间的线路、设备，需要跨越的网络往往是固定的。在我们的互联网上，每时每刻都有大量的应用在互发消息。而这些应用要复用同样的基础建设——网线、路由器、网关、基站等。

应用层的多路复用，如多个请求使用同一个信道并行的传输，实际上是传输层提供的多路复用能力。传输层的多路复用，比如多个 TCP 连接复用一条线路，实际上是网络层在提供多路复用能力。

多路复用让多个信号（例如：请求/返回等）共用一个信道（例如：一个 TCP 连接）。它有两个明显的优势。提升吞吐量。多一个信号被紧密编排在一起（例如：TCP 多路复用节省了多次连接的数据），这样网络不容易空载。多个信号间隔离。信号间并行传输，并且隔离，不会互相影响。

5.9 三次握手

为什么要三次握手

第一次握手：Client 什么都不能确认；Server 确认了对方发送正常。

第二次握手：Client 确认了：自己发送、接收正常，对方发送、接收正常；Server 确认了：自己接收正常，对方发送正常。

第三次握手：Client 确认了：自己发送、接收正常，对方发送、接收正常；Server 确认了：自己发送、接收正常，对方发送接收正常。

所以三次握手就能确认双发收发功能都正常，缺一不可。

SYN，ACK

SYN (synchronous 建立联接)　　ACK(Acknowledgement 确认)

SYN 是 TCP/IP 建立连接时使用的握手信号。在客户机和服务器之间建立正常的 TCP 网络连接时，客户机首先发出一个 SYN 消息，服务器使用 SYN-ACK 应答表示接收到了这个消息，最后客户机再以 ACK 消息响应。这样在客户机和服务器之间才能建立起可靠的 TCP 连接，数据才可以在客户机和服务器之间传递。

5.10 四次挥手

断开一个 TCP 连接则需要“四次挥手”：

客户端-发送一个 FIN，用来关闭客户端到服务器的数据传送

服务器-收到这个 FIN(finish 结束)，它发回一个 ACK，确认序号为收到的序号加 1 。和 SYN 一样，一个 FIN 将占用一个序号

服务器-关闭与客户端的连接，发送一个 FIN 给客户端

客户端-发回 ACK 报文确认，并将确认序号设置为收到序号加 1

为什么要四次挥手

任何一方都可以在数据传送结束后发出连接释放的通知，待对方确认后进入半关闭状态。当另一方也没有数据再发送的时候，则发出连接释放通知，对方确认后就完全关闭了 TCP 连接。

举个例子：A 和 B 打电话，通话即将结束后，A 说“我没啥要说的了”，B 回答“我知道了”，但是 B 可能还会有要说的话，A 不能要求 B 跟着自己的节奏结束通话，于是 B 可能又巴拉巴拉说了一通，最后 B 说“我说完了”，A 回答“知道了”，这样通话才算结束。

5.11 TCP 可靠通信的具体实现

TCP 发送的报文段是交给 IP 层传送的，但 IP 层只能提供尽最大努力交付的服务，也就是说，TCP 下面的网络所提供的是不可靠的传输。因此，TCP 采用了一些适当的措施来提供可靠的传输，使得两个传输层直接的通信变得可靠。

TCP 协议通过使用连续 ARQ 协议和滑动窗口协议，来保证数据传输的正确性，从而提供可靠的传输。

ARQ 协议

ARQ 协议，即自动重传请求（Automatic Repeat-reQuest），是 OSI 模型中数据链路层和传输层的错误纠正协议之一。它通过使用确认和超时这两个机制，在不可靠服务的基础上实现可靠的信息传输。如果发送方在发送后一段时间之内没有收到确认帧，它通常会重新发送。ARQ 包括停止等待 ARQ 协议和连续 ARQ 协议。

停止等待 ARQ 协议

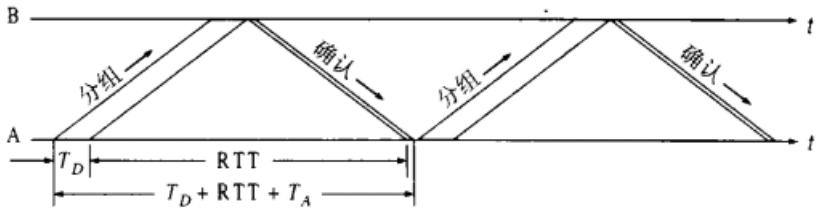
要想弄明白为什么 TCP 要使用连续 ARQ 协议，首先需要弄清楚停止等待 ARQ 协议的原理。

A 发送分组 M1，发送完就暂停发送，等待 B 的确认。B 收到 M1 就向 A 发送确认。A 在收到了对 M1 的确认后，就再发送下一个分组 M2。依次下去发送剩余的数据...

如果 A 发送的过程中出现差错，B 在接收 M1 时检测出了差错，就丢弃 M1，其他什么都不做（也不会通知 A 收到有差错的分组）。又或者 A 传送的过程中分组丢失了，以上这两种情况下，B 不会发送任何信息。

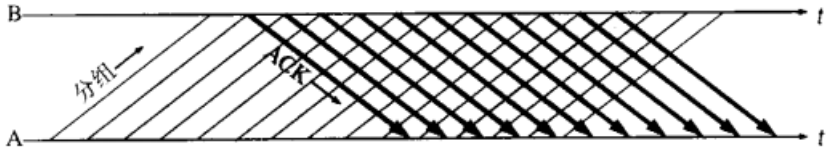
既然说它是可靠传输协议，那自然有它可靠的方法：如果发生以上的情况，A 只要超过了一段时间仍然没有收到确认，就认为刚才发送的分组丢失了，所以它会重传刚刚的发送过的分组，也就是所谓的超时重传。

停止等待 ARQ 协议的优点是简单，但也有很严重的缺点，就是信道利用率太低。



连续 ARQ 协议

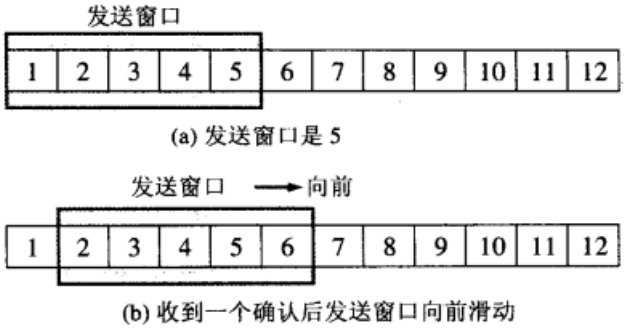
由于停止等待 ARQ 协议信道利用率太低，所以需要使用连续 ARQ 协议来进行改善。这个协议会连续发送一组数据包，然后再等待这些数据包的 ACK。发送方采用流水线传输。流水线传输就是发送方可以连续发送多个分组，不必每发完一个分组就停下来等待对方确认。



连续 ARQ 协议通常是结合滑动窗口协议来使用的，发送方需要维持一个发送窗口，位于发送窗口内的 5 个分组都可以连续发送出去，而不需要等待对方的确认，这样就提高了信道利用率。

连续 ARQ 协议规定，发送方每收到一个确认，就把发送窗口向前滑动一个分组的位置。

接收方一般都是采用累积确认的方式。也就是说接收方不必对收到的分组逐个发送确认。而是在收到几个分组后，对按序到达的最后一个分组发送确认。如果收到了这个分组确认信息，则表示到这个分组为止的所有分组都已经正确接收到了。



5.12 TCP 报文段首部格式

源端口和目的端口：各占 2 个字节，建立端口用

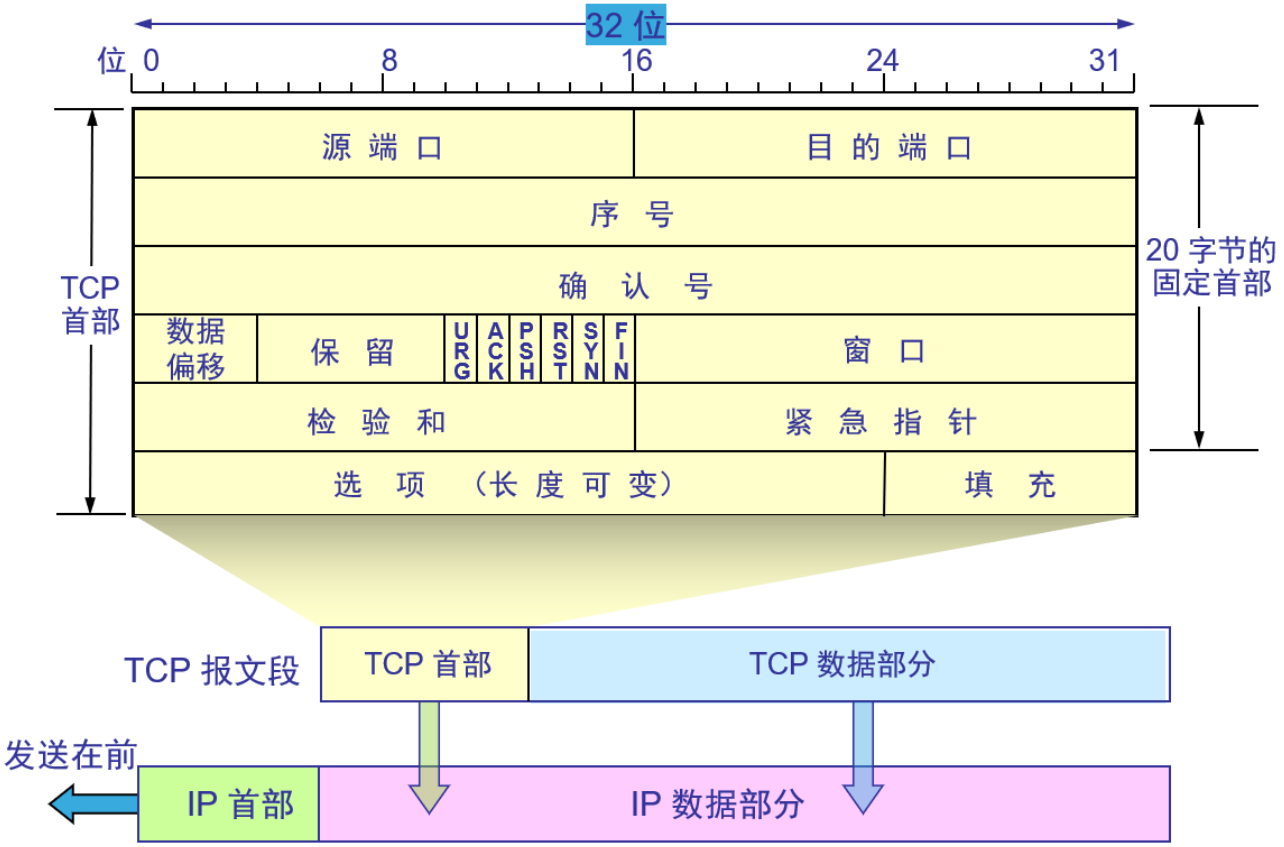
序号：4 个字节，用来给 TCP 传输中的字节流编号，传输的每个字节都要按顺序编号，循环使用。范围是 $[0, 2^{32}-1]$。报文段的序号由要传送的字节路中的第一个字节的序号决定，例如序号字段值为 301，携带 100 字节，那么下一个报文段数据序号从 401 开始，序号字段值为 401

确认号：4 字节长度，期望收到对方下一个报文段的第一个数据字节的序号；例如确认号是 701，那么说明已经接收到了序号到 700 为止的数据

检验和：占 2 字节，检验范围包括首部和数据 2 部分。

紧急指针：占 2 字节，URG=1 时才有用。用来指出本报文段中的紧急数据的字节数

选项：长度可变，最长 40 字节。没有这个的时候 TCP 首部长度为 20 字节



TCP 的六个控制位

紧急位 URG：URG 的特点就是让数据插队，URG=1 的就会在缓存中被提前到第一个传输

确认位 ACK

推送位 PSH：就是接收端的 URG，将 PSH=1 的数据尽快接收

复位 RST：RST=1 时，表明 TCP 连接中出现严重差错，必须释放连接，然后再重新建立传输链接。

同步位 SYN

终止位 FIN

5.13 UDP 的首部格式

源端口：端口号 0-65535，1-1024 保留端口号，为标准的服务端口

目的端口：无须多解释

UDP 长度：header + data 总长度

UDP 校验和：伪头部，头部，data 三部分校验和。

数据：上层应用层的数据。

伪头部：UDP 校验和中的伪头部，并非 UDP 报文中的有效数据，是提取了 IP 数据报中的源 IP，目的 IP 信息并加上协议等字段构造的数据。伪头部在实际网络传输中，仅用作校验和计算使用，并不发送！因此称为伪头部。事实上在 TCP 校验和计算中也用到了伪头部，与 UDP 一致。

5.14 ARP 地址解析协议

ARP 作用：IP 地址解析为 MAC 地址

为什么需要 MAC 地址才能通信？

通过之前的 OSI 七层模型可以知道，数据在数据链路层传输的时候会加一个帧的头部，里面包含 MAC 地址，解封的过程也需要 MAC 地址。

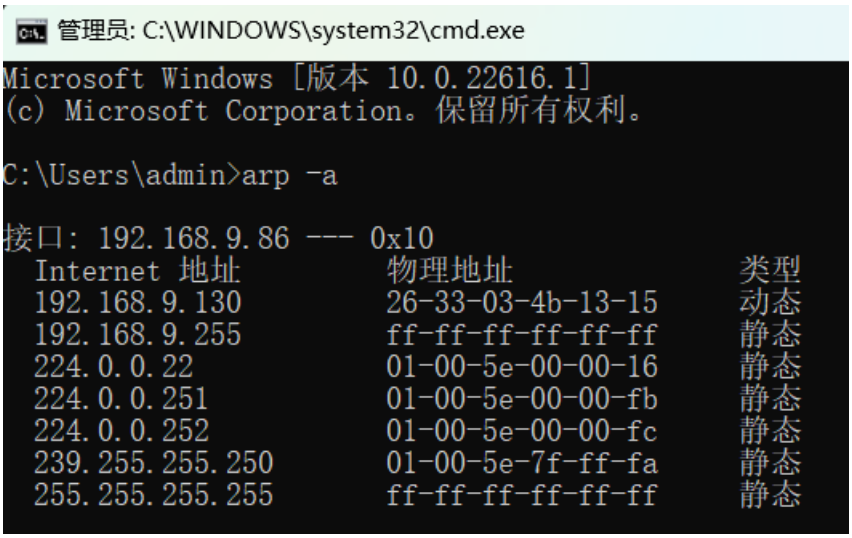


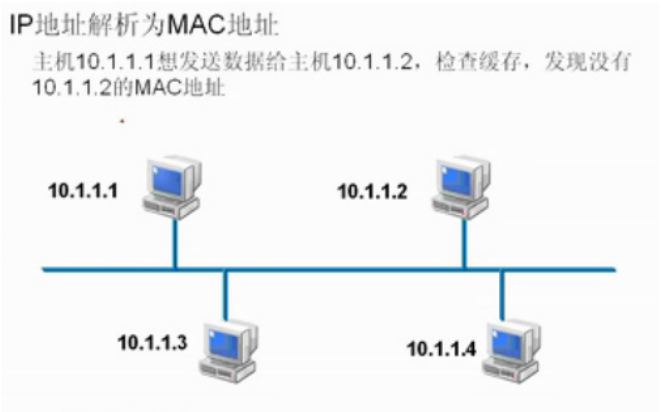
图 5.1: ARP 缓存表 指令 arp -a

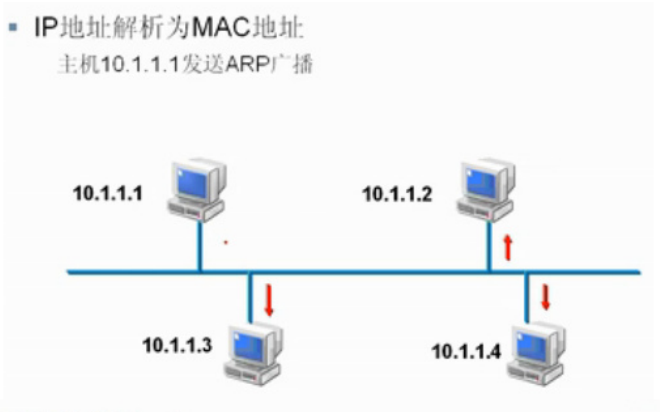
ARP 缓存表是主机最近运行时获得关于其他主机的 IP 地址到 MAC 地址的映射，当需要发送数据的时候，主机就会根据数据报中的目标 IP 地址信息，然后在 ARP 缓存表中进行查找对应的 MAC 地址，最后通过网卡将数据发送出去。

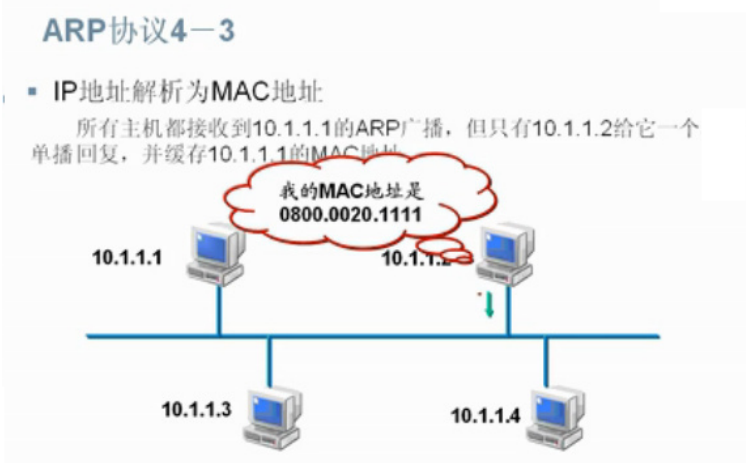
如果没有查到，就向本地网段发起一个 ARP 请求的广播包，查询此目的主机对应的 MAC 地址。此 ARP 请求数据包里包括源主机的 IP 地址、硬件地址、以及目的主机的 IP 地址。网络中所有的主机收到这个 ARP 请求后，会检查数据包中的目的 IP 是否和自己的 IP 地址一致。如果不相同就忽略此数据包；如果相同，该主机首先将发送端的 MAC 地址和 IP 地址添加到自己的 ARP 列表中，如果 ARP 表中已经存在该 IP 的信息，则将其覆盖，然后给源主机发送一个 ARP 响应数据包，告诉对方自己是它需要查找的 MAC 地址；源主机收到这个 ARP 响应数据包后，将得到的目的主机的 IP 地址和 MAC 地址添加到自己的 ARP 列表中，并利用此信息开始数据的传输。如果源主机一直没有收到 ARP 响应数据包，表示 ARP 查询失败。

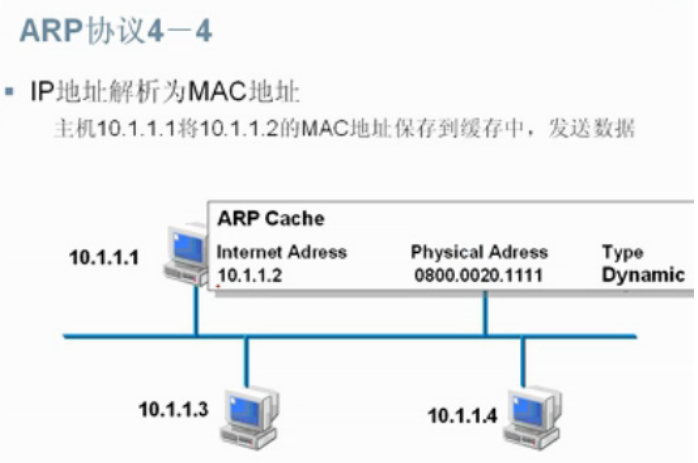
5.15 链路层

物理层解决了信号传输的环境问题，简单理解就是传输了各种信号，但是双方都不能理解对方发来的信号的意思，链路层就是规定一种协议 (信号是谁发的，给谁的，数据部分哪里开始哪里结束的)，使得通信双方能够正常交流。









平常我们说话都是一句一句的，这在链路层中以帧为单位进行传送，另外在通信过程中会产生 (距离远，有人干扰信号) 一些问题，所以引入了透明传输与差错检测两种机制。

MAC 尾部的 FCS 就是差错检测的手段，发往本站的 MAC 帧被收下之后，校验范围从目的地址段到数据段的末尾，算法采用 32 位循环冗余码（CRC），不但需要检验 MAC 帧的数据部分，还要检验目的地址，源地址和类型字段，但不校验前导码。

世界上每块网卡都有一个唯一的地址，即 MAC 地址，因为以太网逻辑上采用总线型拓扑结构，即以太网中所有计算机共享一条总线，信息以广播方式发送，因此网卡从网络上每收到一个 MAC 帧，首先要用硬件检查 MAC 帧中的 MAC 地址，如果是发往本站的帧才会收下，否则丢弃。

5.16 虚拟专用网 VPN

虚拟专用网络 (VPN) 是一门新型的网络技术，它为我们提供了一种通过公用网络 (如最大的公用因特网) 安全地对企业内部专用网络进行远程访问的连接方式。

我们知道一个网络连接通常由三个部分组成：客户机、传输介质和服务器。VPN 网络同样也需要这三部分，不同的是 VPN 连接不是采用物理的传输介质，而是使用一种称之为“隧道”的东西来作为传输介质的，这个隧道是建立在公共网络或专用网络基础之上的，如因特网或专用 Intranet 等。

要实现 VPN 连接，企业内部网络中必须配置有一台 VPN 服务器，VPN 服务器一方面连接企业内部专用网络 (LAN)，另一方面要连接到因特网或其它专用网络，这就要 VPN 服务器必须拥有一个公用的 IP 地址，也就是说企业必须先拥有一个合法的 Internet 或专用网域名。

整个 VPN 通信过程可以简化为以下 4 个通用步骤：

(1) 客户机向 VPN 服务器发出请求；

(2) VPN 服务器响应请求并向客户机发出身份质询，客户机将加密的用户身份验证响应信息发送到 VPN 服务器；

(3) VPN 服务器根据用户数据库检查该响应，如果账户有效，VPN 服务器将检查该用户是否具有远程访问权限；如果该用户拥有远程访问的权限，VPN 服务器接受此连接；

(4) 最后 VPN 服务器将在身份验证过程中产生的客户机和服务器公有密钥将用来对数据进行加密，然后通过 VPN 隧道技术进行封装、加密、传输到目的内部网络。

第 6 章 计算思维与算法

6.1 什么是计算思维？ . . . 45
6.1.1 思考，创新，创造 . . . 45
6.1.2 菜谱 . . . 46
6.1.3 求解问题中的计算思维 . . . 46
6.1.4 系统设计中的计算思维 . . . 46
6.1.5 人类行为理解中的计算思维 . . . 46
6.2 计算思维的本质 . . . 46
6.2.1 理解计算思维，首先要理解计算 . . . 47
6.2.2 抽象 . . . 47
6.3 计算思维对其他学科的影响 . . . 48
6.3.1 人类基因组计划 . . . 48
6.3.2 霰弹枪测序法 . . . 48
6.4 算法的表示 . . . 48
6.4.1 自然语言表示 . . . 48
6.4.2 流程图 . . . 48
6.5 迭代算法 . . . 49
6.5.1 迭代法求平方根 . . . 49
6.6 递归算法 . . . 50
6.6.1 二分搜索递归算法 . . . 51
6.7 分治算法 . . . 52
6.8 贪心算法 . . . 53
6.8.1 贪心算法思想 . . . 53
6.8.2 贪心算法的基本要素 . . . 53
6.8.3 贪心算法的基本思路 . . . 53
6.8.4 算法特性 . . . 54
6.8.5 海盗船装载问题 . . . 54
6.9 动态规划 . . . 55
6.10 回溯法 . . . 59
6.10.1 回溯法算法思想 . . . 59
6.10.2 问题的解空间和空间树 . . . 59
6.11 算法实例 . . . 60

6.1 什么是计算思维？

计算思维是一种人类的思维过程，是每个人必须掌握的技能。与“听说读写”能力一样，是人类基本思维方式。计算思维不仅仅属于计算机科学家，而是属于每一个人。

6.1.1 思考，创新，创造

当今世界已然是科技驱动的社会。谁拥有核心科技硬实力，谁就能掌握主动。从国家到个人，均无例外。以往教学的重点常常集中于如何让学生理解、记忆现有的体系知识。而世界信息体系发生了变化，我们的教学不可一

成不变，也应该随之变化。不能再只是让孩子们了解已有的这些信息（数据），而应该教会他们分析、处理及展示这些信息（数据），把它们真正地用起来。如今的孩子们不应该再过多地死记硬背已有的知识，而应更多的去思考，去创新，去创造。孩子们要避免形成思维定势，多锻炼将所学应用到现实世界的问题中的能力。

6.1.2 菜谱

人们在日常生活中的很多做法其实都和计算思维不谋而合，也可以说计算思维从生活中吸收了很多有用的思想和方法。我们来看一些例子。

算法过程：菜谱可以说是算法（或程序）的典型代表，它将一道菜的烹饪方法一步一步地罗列出来，即使不是专业厨师，照着菜谱的步骤也能做出可口的菜肴。这里，菜谱的每一步骤必须足够简单、可行。例如："将土豆切成块状"、"将 1 两油入锅加热"等都是可行的步骤，而"使菜肴具有神秘香味"则不是可行的。

模块化：很多菜谱都有"勾芡"这个步骤，与其说这是一个基本步骤，不如说是一个模块，因为勾芡本身代表着一个操作序列——取一些淀粉，加点水，搅拌均匀，在适当时候倒入菜中。由于这个操作序列经常使用，为了避免重复，也为了使菜谱结构清晰、易读，所以用"勾芡"这个术语简明地表示。这个例子同时也反映了在不同层次上进行抽象的思想。

并发。厨师在烧菜时，如果一个菜需要在锅中煮一段时间，厨师一定会利用这段时间去做点别的事情（比如将另一个菜洗净切好），而绝不会无所事事。在此期间如果锅里的菜需要加盐加佐料，厨师可以放下手头的活儿去处理锅里的菜。就这样，虽然只有一个厨师，但他可以同时做几个菜。

6.1.3 求解问题中的计算思维

利用计算手段求解问题的过程是：首先要把实际的应用问题转换为数学问题，可能是一组偏微分方程，其次将 PDE 离散为一组代数方程组，然后建立模型、设计算法和编程实现，最后在实际的计算机中运行并求解。前两步是计算思维中的抽象，后两步是计算思维中的自动化。

6.1.4 系统设计中的计算思维

R.Karp：任何自然系统和社会系统都可视为一个动态演化系统，演化伴随着物质、能量和信息的交换，这种交换可以映射为符号变换，使之能用计算机进行离散的符号处理。

当动态演化系统抽象为离散符号系统后，就可以采用形式化的规范描述，建立模型、设计算法和开发软件来揭示演化的规律，实时控制系统的演化并自动执行。

6.1.5 人类行为理解中的计算思维

利用计算手段来研究人类的行为，可视为社会计算，即通过各种信息技术手段，设计、实施和评估人与环境之间的交互。

6.2 计算思维的本质

Abstraction （抽象）和 Automation（自动化）是计算思维的两大核心特征。

1. 把实际问题抽象为数学问题，并建模
   将人对问题的理解用数学语言描述出来
2. 进行映射，把数学模型中的变量等用特定的符号代替
   用符号一一对应数学模型中的变量和规则等
3. 通过编程把解决问题的逻辑分析过程写成算法
   把解题思路变成计算机指令，也就是算法

4. 执行算法，进行求解

计算机根据算法，一步步完成相应指令，求出结果

建立数学模型的过程就是理解问题的过程，并且要把你对问题的理解用数学语言描述出来。这很关键，数学模型的好坏意味着你对问题的理解程度够不够深，而且数学模型还说明了在这个问题中，哪些东西可以计算以及如何进行计算，这可以说是计算思维里最最核心的东西了。这个关键过程需要的核心能力就是抽象能力以及一定的数学基础。

数学建模只是可计算化的第一步，为了让计算机帮我们去求解，我们还需要虚拟的符号来代替的数学模型里的每个变量和运算规则，这个过程就是映射啦！

完成映射，我们就能把解题思路（注意，是解题思路，不是数学模型）用程序语言完整地告诉计算机啦，这个过程就是具体的编程写算法的过程啦！这一步需要较强的编程能力，但编程能力的核心之一也是抽象思维能力。对于编程能力不够强的人来说，映射还有编程的过程可以交给擅长编程的人来做。

关键路径的前 3 步都是人来完成的，最后一步执行算法进行运算是机器自动完成的，体现了计算思维的自动化的特点。

在整个过程中，抽象是方法，是手段，贯穿整个过程的每个环节。自动化是最终目标，让机器去做计算的工作，把人脑解放出来，中间目标是实现问题的可计算化，体现在成果上就是数学模型、映射、还有算法。

6.2.1 理解计算思维，首先要理解计算

理解计算思维的前提是理解计算，因为计算思维本质上还是研究计算的，研究在解决问题过程中，哪些是可计算的，以及如何计算。

通常我们理解的计算是算术运算，如“1+1=2”,，但运算其实有很多种类，如集合运算、逻辑运算、条件运算等等。集合运算这里面就没有具体的数值运算了，而是用代表集合的字母进行运算；又比如逻辑运算“1□0=0”，这个运算里有数值“0”和 1，但意义完全不同，这里的“1”代表的是“真”—即命题为真，“0”代表的是“假”—即命题为假，通过用数字“0”和“1”来代换命题的真假，用“□”来代换逻辑语言里的“并且”，逻辑判断过程也能通过计算来实现。

当我们跳出算术运算的局限，理解了计算的本质后，就会发现原来好多看似不可计算的东西都能变得可计算，也就很容易理解计算思维的普适性了。因为经过一定的抽象，我们对很多问题的理解都能用特定的数学语言来描述，接下来，当我们用特定的数学语言去描述解决过程的时候，就是在用计算化的方式来求解了。

6.2.2 抽象

在哥尼斯堡城的普莱格尔河上有 7 座桥，将河中的两个岛和河岸连结问能否一次走遍 7 座桥，而每座桥只允许通过一次，最后仍然回到起始地点。

1736 年 29 岁的欧拉向圣彼得堡科学院递交了《哥尼斯堡的七座桥》的论文，在解答问题的同时，开创了数学的一个新的分支——图论与几何拓扑，也由此展开了数学史上的新历程。

大数学家欧拉把它转化成一个几何问题——一笔画问题。他不仅解决了此问题，且给出了连通图可以一笔画的充要条件是：奇点的数目不是 0 个就是 2 个（连到一点的数目如果是奇数条，就称为奇点；如果是偶数条，就称为偶点。要想一笔画成，必须中间点均是偶点，也就是有来路必有另一条去路，奇点只可能在两端。因此任何图能一笔画成，奇点要么没有，要么在两端）

6.3 计算思维对其他学科的影响

6.3.1 人类基因组计划

人类基因组计划（英语：Human Genome Project, HGP）是一项规模高，跨国跨学科的科学探索巨型工程。其宗旨在于测定组成人类染色体（指单倍体）中所包含的六十亿对组成的核苷酸序列，从而绘制人类基因组图谱，并且辨识其载有的基因及其序列，达到破译人类遗传信息的最终目的。基因组计划是人类为了探索自身的奥秘所迈出的重要一步。截至 2005 年，人类基因组计划的测序工作已经基本完成（92%）。其中，2001 年人类基因组工作草图的发表（由公共基金资助的国际人类基因组计划和私人企业塞雷拉基因组公司各自独立完成，并分别公开发表）被认为是人类基因组计划成功的里程碑。

6.3.2 霰弹枪测序法

霰弹枪定序法（又称鸟枪法）是一种广泛使用的为长 DNA 测序的方法，比传统的定序法快速，但精确度较差。曾经使用于塞雷拉基因组（Celera Genomics）公司所主持的人类基因组计划。

霰弹枪测序法的思想是将基因组打断为数百万个 DNA 片断，然后用一定的算法将片断的序列信息重新整合在一起，从而得到整个基因组序列。为了提高这一方法的效率，1980 年代，测序和片断信息整合达到了自动化。这一方法虽然已被用于序列长达 6 百万个碱基对的细菌基因组测序，但对于人类基因组中 3 千万个碱基对的序列测定，这一技术能否成功在当时还未有定论。

6.4 算法的表示

6.4.1 自然语言表示

用自然语言来描述算法。

使用自然语言描述算法的优点是通俗易懂。但是，自然语言本身所固有的不严密性使得这种描述方法存在“文字冗长，容易产生歧义性”以及“难以描述算法中的分支和循环等结构”等缺陷。

6.4.2 流程图

流程图是一种传统的、广泛应用的算法描述工具，也是最常见的算法图形化表达工具。

流程图利用几何图形的图框来代表各种不同的操作，用流程线来指示算法的执行方向，它使用规定的一些图框、线条来形象、直观地描述算法处理过程。

与自然语言相比，流程图可以清晰、直观、形象地反应控制结构的过程。

为什么使用变量？

存放数据。

算法中的初始数据、计算的中间结果和最后结果都需要存放到变量中。

如何在变量中存入数据？

- 赋值
- 输入

大多算法和编程语言使用等号 = 表示赋值操作，称为赋值号。等号在算法中表示的是赋值，不是相等。

例如：

$a = 5$

图示	名称	说明
	开始/结束框	算法的开始或结束
	输入/输出框	变量的输入或输出
	处理框	变量的计算与赋值
	判断框	条件判断
→	流程线	算法的流向
○	连接点	算法的连接

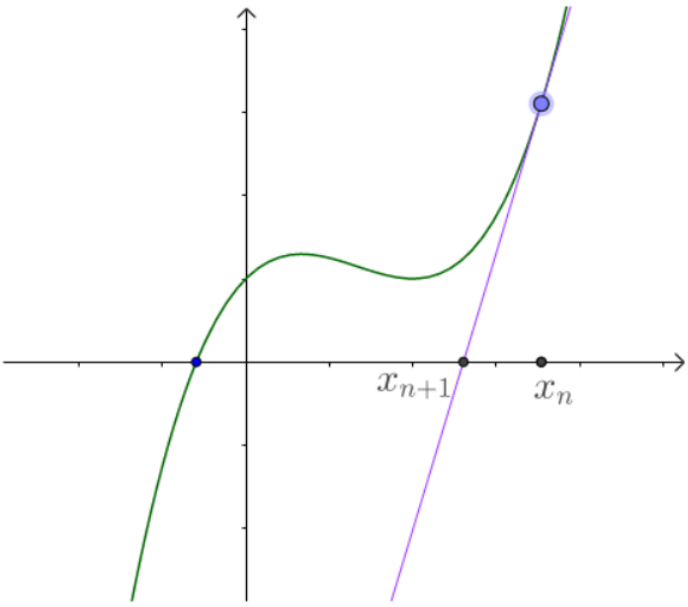
$a = a + 3$

如果表示数学上的等于，这是不合理的！而算法中表示的是赋值：先计算加，读取 a 的值 5 加 3 得 8，然后将 8 赋值给 a，a 的值变为 8。

每次对变量赋值时，新值会覆盖掉变量中原来存放的值。所以在一段时间中变量的值是变化的，但在任一时刻只对应一个值。

6.5 迭代算法

6.5.1 迭代法求平方根



原理：

如图，一个曲线方程 $f(x)$，在它的 $f(x_n)$ 处画一条切线与 x 轴相交，交点为 x_{n+1}，如果继续在它的 $f(x_{n+1})$ 处画一条切线与 x 轴相交，会得到交点 x_{n+2}。而在这个过程中，可以发现交点 x_{n+m} 会无限逼近方程 $f(x) = 0$ 的解，最终可以得到一个与理想值无限靠近的解。

而这里讨论的是求平方根，所以曲线方程更简单。比如，我们要求 N 的平方根。

那么其实就是求方程 $f(x) = x^2 - N$，当 $f(x) = 0$ 时方程的解。

函数 $f(x)$ 的导函数是：$f'(x) = 2x$

那么 $f(x)$ 函数的曲线在 $(x_n, x_n^2 - N)$ 点处切线的斜率为：$2x_n$

所以切线方程为：$f(x) - (x_n^2 - N) = 2x_n(x - x_n)$，即：$f(x) = 2x_n x - x_n^2 - N$

那么切线方程与 x 轴的交点 $x_{n+1} = (x_n + N/x_n)/2$，

我们可以将得到的交点值的平方与 N 比较，循环以上过程直到得到满意的值。

6.6 递归算法

一个函数在它的函数体内调用它自身称为递归调用，这种函数称为递归函数。执行递归函数将反复调用其自身，每调用一次就进入新的一层，当最内层的函数执行完毕后，再一层一层地由里到外退出。

阶乘 $n!$ 的计算公式如下：

$$n! = \begin{cases} 1 & (n = 0, 1) \\ n * (n-1)! & (n > 1) \end{cases}$$

根据公式编写如下的代码：

```
n = int(input("请输入n: "))
def factorial(n):
if n == 0:
    return 1
return n * factorial(n - 1)
print(factorial(n))
```

factorial() 就是一个典型的递归函数。调用 factorial() 后即进入函数体，只有当 n==0 或 n==1 时函数才会执行结束，否则就一直调用它自身。

由于每次调用的实参为 n-1，即把 n-1 的值赋给形参 n，所以每次递归实参的值都减 1，直到最后 n-1 的值为 1 时再作递归调用，形参 n 的值也为 1，递归就终止了，会逐层退出。

要想理解递归函数，重点是理解它是如何逐层进入，又是如何逐层退出的，下面我们以 5! 为例进行讲解。

递归的进入

1. 求 5!，即调用 factorial(5)。当进入 factorial() 函数体后，由于形参 n 的值为 5，不等于 0 或 1，所以执行 factorial(n-1) * n，也即执行 factorial(4) * 5。为了求得这个表达式的结果，必须先调用 factorial(4)，并暂停其他操作。换句话说，在得到 factorial(4) 的结果之前，不能进行其他操作。这就是第一次递归。
2. 调用 factorial(4) 时，实参为 4，形参 n 也为 4，不等于 0 或 1，会继续执行 factorial(n-1) * n，也即执行 factorial(3) * 4。为了求得这个表达式的结果，又必须先调用 factorial(3)。这就是第二次递归。
3. 以此类推，进行四次递归调用后，实参的值为 1，会调用 factorial(1)。此时能够直接得到常量 1 的值，并把结果 return，就不需要再次调用 factorial() 函数了，递归就结束了。

逐层进入的过程

层次/层数	实参/形参	调用形式	需要计算的表达式	需要等待的结果
1	n=5	factorial(5)	factorial(4) * 5	factorial(4) 的结果
2	n=4	factorial(4)	factorial(3) * 4	factorial(3) 的结果
3	n=3	factorial(3)	factorial(2) * 3	factorial(2) 的结果
4	n=2	factorial(2)	factorial(1) * 2	factorial(1) 的结果
5	n=1	factorial(1)	1	无

递归的退出

当递归进入到最内层的时候，递归就结束了，就开始逐层退出了，也就是逐层执行 return 语句。

1. n 的值为 1 时达到最内层，此时 return 出去的结果为 1，也即 factorial(1) 的调用结果为 1。
2. 有了 factorial(1) 的结果，就可以返回上一层计算 factorial(1) * 2 的值了。此时得到的值为 2，return 出去的结果也为 2，也即 factorial(2) 的调用结果为 2。
3. 以此类推，当得到 factorial(4) 的调用结果后，就可以返回最顶层。经计算，factorial(4) 的结果为 24，那么表达式 factorial(4) * 5 的结果为 120，此时 return 得到的结果也为 120，也即 factorial(5) 的调用结果为 120，这样就得到了 5! 的值。

逐层退出的过程

层次 层数	调用形式	需要计算 的表达式	从内层递归得到的结果 （内层函数的返回值）	表达式的值 （当次调用的结果）
5	factorial(1)	1	无	1
4	factorial(2)	factorial(1) * 2	factorial(1) 的返回值，也就是 1	2
3	factorial(3)	factorial(2) * 3	factorial(2) 的返回值，也就是 2	6
2	factorial(4)	factorial(3) * 4	factorial(3) 的返回值，也就是 6	24
1	factorial(5)	factorial(4) * 5	factorial(4) 的返回值，也就是 24	120

递归的条件

每一个递归函数都应该只进行有限次的递归调用，否则它就会进入死胡同，永远也不能退出了，这样的程序是没有意义的。

要想让递归函数逐层进入再逐层退出，需要解决两个方面的问题：

1. 存在限制条件，当符合这个条件时递归便不再继续。对于 factorial()，当形参 n 等于 0 或 1 时，递归就结束了。
2. 每次递归调用之后越来越接近这个限制条件。对于 factorial()，每次递归调用的实参为 n - 1，这会使得形参 n 的值逐渐减小，越来越趋近于 1 或 0。

6.6.1 二分搜索递归算法

如何在数组中查找某一特定元素?

最 low 的算法：枚举

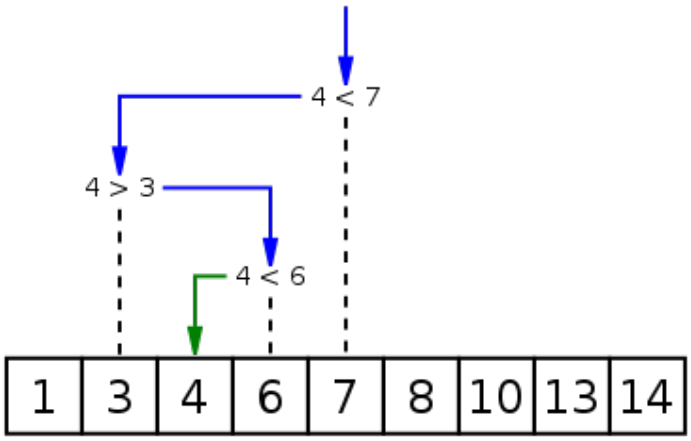
```
lis = [1, 3, 4, 6, 7, 8, 10, 13, 14, 20, 22, 24, 30, 50, 80, 98, 100]
n = int(input("输入要查找的数:"))
for item in lis:
    if n == item:
        print("找到了")
        break
```

二分查找是一种在有序数组中查找某一特定元素的搜索算法。搜索过程从数组的中间元素开始，如果中间元素正好是要查找的元素，则搜索过程结束；如果某一特定元素大于或者小于中间元素，则在数组大于或小于中间元素的那一半中查找，而且跟开始一样从中间元素开始比较。如果在某一步骤数组为空，则代表找不到。这种搜索算法每一次比较都使搜索范围缩小一半。

```
lis = [1, 3, 4, 6, 7, 8, 10, 13, 14, 20, 22, 24, 30, 50, 80, 98, 100]

n = int(input('输入要查找的数:'))
left = 0
right = len(lis) - 1

```




```
while left <= right:
    mid = (left + right) // 2 # 整除
    if n > lis[mid]:
        left = mid + 1
    elif n < lis[mid]:
        right = mid - 1
    else:
        print("要查找的数在数组中的索引为%s" % mid)
        break
else:
    print("没有这个数据")
```

6.7 分治算法

分治算法的思想

将问题规模为 n 的原问题归纳为规模 n/2 的两个子问题，或规模 n/m 的 m 个子问题；若规模缩小的子问题仍不能简单求解，则继续归约为规模更小的问题；分别求解小规模的子问题，并逐步综合得到原问题的解。

分治法所能解决的问题一般具有以下几个特征：

1. 该问题的规模缩小到一定的程度就可以容易地解决
2. 该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质。
3. 利用该问题分解出的子问题的解可以合并为该问题的解；
4. 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子子问题。

第一条特征是绝大多数问题都可以满足的，因为问题的计算复杂性一般是随着问题规模的增加而增加；

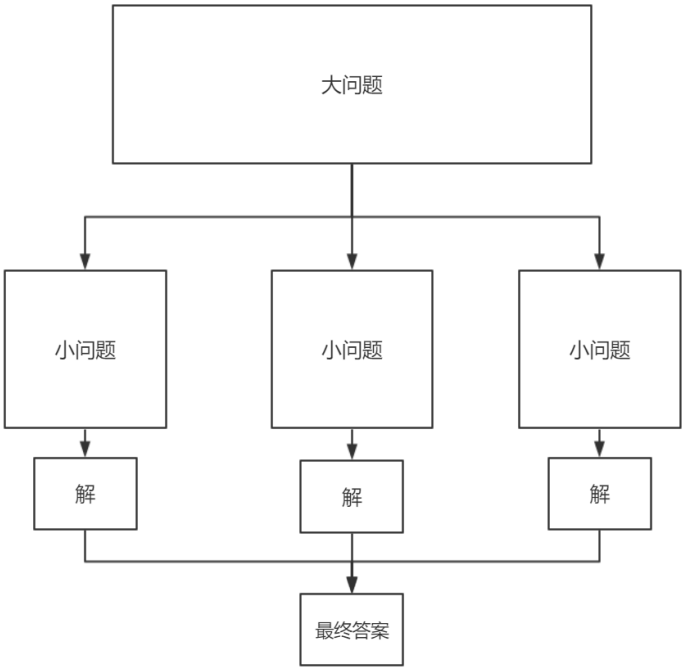
第二条特征是应用分治法的前提它也是大多数问题可以满足的，此特征反映了递归思想的应用；

第三条特征是关键，能否利用分治法完全取决于问题是否具有第三条特征，如果具备了第一条和第二条特征，而不具备第三条特征，则可以考虑用贪心法或动态规划法。

第四条特征涉及到分治法的效率，如果各子问题是不独立的则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然可用分治法，但一般用动态规划法较好。

分治算法的解题步骤一般如下：

1. 分解，将要解决的问题划分成若干规模较小的同类问题；
2. 求解，当子问题划分得足够小时，用较简单的方法解决；
3. 合并，按原问题的要求，将子问题的解逐层合并构成原问题的解。



6.8 贪心算法

6.8.1 贪心算法思想

顾名思义，贪心算法总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的局部最优选择。当然，希望贪心算法得到的最终结果也是整体最优的。虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路经问题，最小生成树问题等。在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。

6.8.2 贪心算法的基本要素

1. 贪心选择

   所谓贪心选择是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。

   动态规划算法通常以自底向上的方式解各子问题，而贪心算法则通常以自顶向下的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

   对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

2. 最优子结构

   当一个问题的最优解包含其子问题的最优解时，称此问题具有最优子结构性质。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。

   贪心算法的每一次操作都对结果产生直接影响，而动态规划则不是。贪心算法对每个子问题的解决方案都做出选择，不能回退；动态规划则会根据以前的选择结果对当前进行选择，有回退功能。动态规划主要运用于二维或三维问题，而贪心一般是一维问题。

6.8.3 贪心算法的基本思路

贪心算法的基本思路是从问题的某一个初始解出发一步一步地进行，根据某个优化测度，每一步都要确保能获得局部最优解。每一步只考虑一个数据，他的选取应该满足局部优化的条件。若下一个数据和部分最优解连在

一起不再是可行解时，就不把该数据添加到部分解中，直到把所有数据枚举完，或者不能再添加算法停止。

1. 建立数学模型来描述问题；
2. 把求解的问题分成若干个子问题；
3. 对每一子问题求解，得到子问题的局部最优解；
4. 把子问题的解局部最优解合成原来解问题的一个解。

6.8.4 算法特性

- 随着算法的进行，将积累起其它两个集合：一个包含已经被考虑过并被选出的候选对象，另一个包含已经被考虑过但被丢弃的候选对象。
- 有一个函数来检查一个候选对象的集合是否提供了问题的解答。该函数不考虑此时的解决方法是否最优。
- 还有一个函数检查是否一个候选对象的集合是可行的，也即是否可能往该集合上添加更多的候选对象以获得一个解。和上一个函数一样，此时不考虑解决方法的最优性。
- 选择函数可以指出哪一个剩余的候选对象最有希望构成问题的解。
- 最后，目标函数给出解的值。
- 为了解决问题，需要寻找一个构成解的候选对象集合，它可以优化目标函数，贪婪算法一步一步的进行。起初，算法选出的候选对象的集合为空。接下来的每一步中，根据选择函数，算法从剩余候选对象中选出最有希望构成解的对象。如果集合中加上该对象后不可行，那么该对象就被丢弃并不再考虑；否则就加到集合里。每一次都扩充集合，并检查该集合是否构成解。如果贪婪算法正确工作，那么找到的第一个解通常是最优的。

6.8.5 海盗船装载问题

（1）有一天海盗们截获了一艘装满各种各样宝物的货船，每一件都价值连城，海盗船载重量为 w，每件宝物的重量为 w_i，海盗们该如何尽可能装载最多数量的宝物呢？（2）每种宝物价值 v_i，宝物可分割。怎样才能使运走宝物的价值最大呢？

宝物清单								
重量	4	10	7	11	3	5	14	2
价值	8	50	14	44	9	20	28	6

```python
goods = [4, 10, 7, 11, 3, 5, 14, 2]
anti_sort = sorted(goods) # 对重量排序
w = int(input("请输入海盗船所能装载的最大容量:"))
m = 0
tmp = 0 # m记录装载古董数量, tmp记录装载古董重量
ship = [] # 记录装载的古董
for a in anti_sort:
    tmp = tmp + a
    if tmp <= w:
        m = m + 1
        ship.append(a)
print("装载古董的数量:", m)
print("装载的古董", ship)
```

```python
# 每个商品元组表示（价格，重量）
goods = [(8, 4), (50, 10), (14, 7), (44, 11), (9, 3), (20, 5), (28, 14), (6, 2)]
```

```python
# 对商品按单位价值进行排序
goods.sort(key=lambda x: x[0] / x[1], reverse=True)
print(goods)

# w 表示背包的容量
# m 表示每个商品拿走多少个
def fractional_backpack(goods, w):
    total_v = 0
    m = [0 for _ in range(len(goods))]
    for i, (price, weight) in enumerate(goods):
        if w >= weight:
            m[i] = 1
            total_v += price
            w -= weight
            # m[i] = 1 if w>= weight else weight / w
        else:
            m[i] = w / weight
            total_v += m[i] * price
            w = 0
            break
    return m, total_v

res1, res2 = fractional_backpack(goods, 30)
print(res1, res2)
```

6.9 动态规划

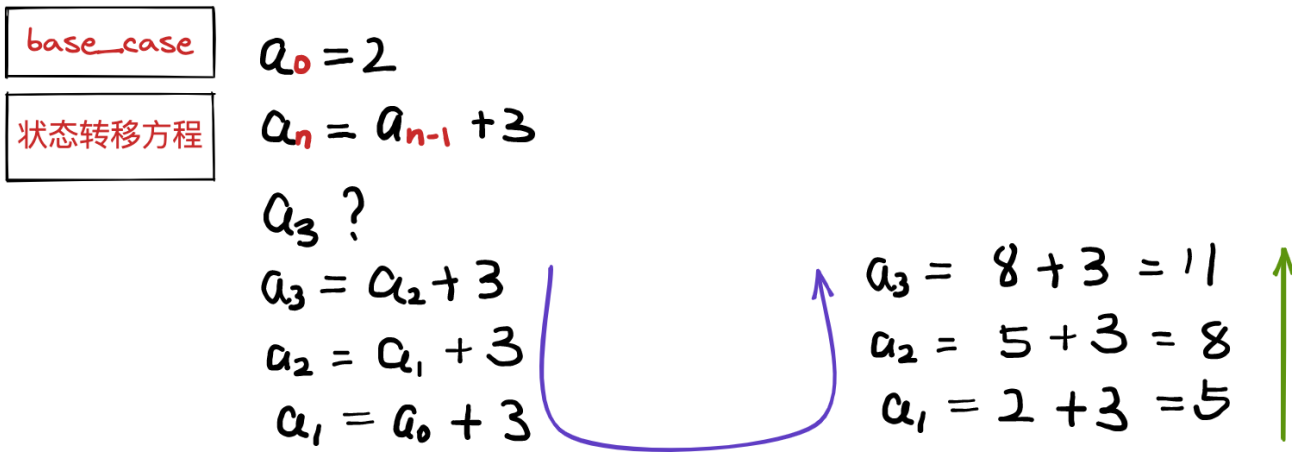
定义

动态规划其实是一种运筹学方法，是在多轮决策过程中寻找最优解的方法。

应用场景

动态规划问题的一般形式就是求最值。动态规划其实是运筹学的一种最优化方法，只不过在计算机问题上应用比较多，比如说求最长递增子序列，最小编辑距离等等。

要理解动态规划的基本解题思路，先来做一道高中数学题。



通俗来讲动态规划算法并不直接给出最终结果的求解表达式，而是通过找到问题规模之间的状态转移方程，借此不断缩小问题规模，逐渐迫近base case。所谓动态转移方程就是不同规模问题之间的关系，类似递推式。

动态规划解题方式

动态规划的解题方式通常分为两种：

1. 通过定义递归方程解决，这是一种自顶向下的求解方式，通常这种方式会有很多重复计算过程，因此可以通过建立备忘录记录中间过程来进行优化。如上图紫色箭头方向。

```
def func(n):
    if n == 0:
        return 2
    return func(n - 1) + 3
```

2. 通过定义 DP(Dynamic Programming) 数组来求解，这是一种自底向上的求解方式。如上图绿色箭头方向。

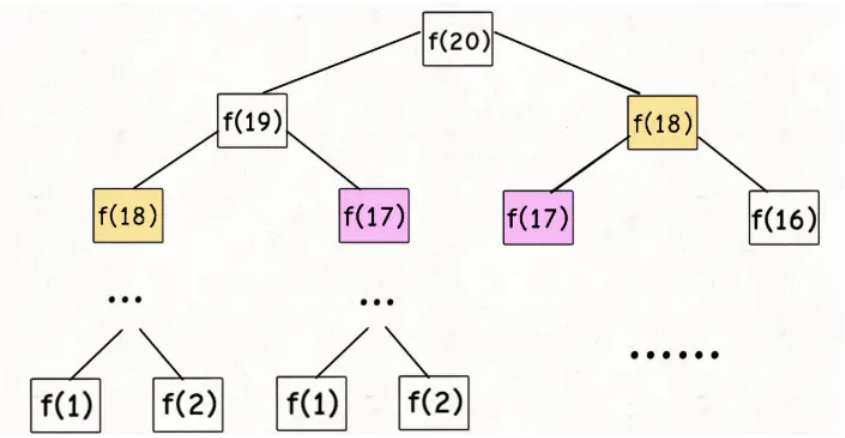
```
def func2(n):
    # 确定dp[i] 的含义
    # d[i] == func(i)
    # dp数组规模,规模与dp[i]的定义密切相关
    # 根据dp[i] 定义，func2(n) 对应dp[n]
    # 要使得dp[n] 合法，数组长度要为 n+1
    dp = [0] * (n + 1)
    # base_case
    dp[0] = 2
    # dp数组的遍历顺序
    # func(i) = func(i-1) + 3
    # dp[i] = dp[i-1] + 3
    for i in range(1, n + 1):
        dp[i] = dp[i - 1] + 3
    return dp[n]
```

重叠子问题

以斐波拉契数列为例：

```
def fib(n):
    if n <= 2:
        return 1
    return fib(n-1) + fib(n-2)
```

我们来看一下递归的状态树：



从状态树我们可以看到有很多重复计算的节点，重叠子问题导致了大量的重复计算。并且重复的计算量随着

问题规模增大极速上升。此时的时间复杂度 $O(2^n)$ 指数级，计算很慢。

因此，使用备忘录来减少重复计算非常有必要！我们可以通过建立一个备忘录 memo，来记录中间节点的计算结果，避免重复计算，可以将时间复杂度直接降为 O(n) 线性复杂度，优化代码如下：

```
# 使用一个字典存储计算过的fib(n)
# base_case 也可直接放入备忘录
# 字典以 n为键，fib(n) 为值
memo = {0: 0, 1: 1}

def fib(n):
    if n in memo:
        return memo[n]
    memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]
```

备忘录相当于为整个递归遍历树进行了一个剪枝操作。

使用 dp 数组自底向上来解不存在重叠子问题。空间开销为 O(n)

```
def fib(n):
    dp = [0 for _ in range(n+1)]
    # 基础值，也就是baseline
    dp[1] = dp[2] = 1
    for i in range(3, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

不难看出，每次计算新值只依赖前面两个值。因此我们可以使用两个变量来记录，而不使用 dp 数组，通过状态压缩减少空间开销。

```
def fib(n):
    if n <= 2:
        return 1
    prev = 1
    curr = 1
    for i in range(3, n+1):
        sum = prev + curr
        prev = curr
        curr = sum
    return curr
```

当出现重叠子问题时，自顶向下的解法一需要备忘录来剪枝，自底向上的解法二需要状态压缩减少空间开销。一般来说，自顶向下的方法如果不用备忘录剪枝，一般会超时。自底向上的方法不进行状态压缩一般也没事。

通用解题过程

动态规划没有标准的解题方法，即没有一个完全通用的解题方法。但在宏观上有通用的方法论：

下面的 k 表示多轮决策的第 k 轮：

1. 问题分解，将原问题划分成几个子问题。一个子问题就是多轮决策的一个阶段。
2. 找状态，选择合适的状态变量 S(k)。它需要具备描述多轮决策过程的演变，更像是决策可能的结果。
3. 做决策，确定决策变量 u(k)。每一轮的决策就是每一轮可能的决策动作，即当前可以有哪些决策可以选择。
4. 状态转移方程。这个步骤是动态规划最重要的核心，即 S(k+1)= uk(sk) 。

5. 定目标。写出代表多轮决策目标的指标函数 V(k,n)，即最终需要达到的目标。
6. 寻找目标的终止条件。

状态转移方程

动态规划的解题核心就是找到状态转移方程，其实所谓的状态转移方程说白了就是数学上的递推方程，没有什么高大上的。就是通过最基础的问题，一步步递推出所需要的最终目标结果。只是在定义状态转移方程的含义时，有不同的定义方式，不同的定义方式会有不同的递推方程式，但是只要定义正确，都可以得到最终正确的结果。

通常状态转移方程定义过程：

1. 明确当前可改变的状态有什么；
2. 定义 dp 数组或者递归函数的具体含义；
3. 明确每一步可以进行的选择有哪些；
4. 编写递推方程，也就是状态转移方程；
5. 明确 baseline，也就是递推开始时的基础值是什么。

海盗船装载问题

有一天海盗们截获了一艘装满各种各样宝物的货船，每一件都价值连城，宝物可分割。海盗船载重量为 w，每件宝物的重量为 w_i，价值 v_i，海盗们该如何装载宝物，使运走宝物的价值最大呢？

宝物清单								
重量	4	10	7	11	3	5	14	2
价值	8	50	14	44	9	20	28	6

记 $m(i,w)$：表示载重量为 w，现有 i 件宝物可装载，所能得到的最大价值。

- 如果第 i 件宝物太重，装不到船上，那么前 i 件宝物价值等于前 i-1 件宝物价值，即 $m(i,w)=m(i-1,w)$；
- 如果第 i 件宝物可以加入到背包中，那么前 i 件宝物价值等于前 i-1 件宝物价值加上第 i 件宝物价值 w_i，即 $m(i,w)=m(i-1,w-w_i)+v_i$。

因此，$m(i,w)$ 应该是 $m(i-1,w)$ 和 $m(i-1,w-w_i)+v_i$ 两者最大值，即

$m(i,w)=\max\{m(i-1,w),m(i-1,w-w_i)+v_i\}$

$$m(i,W)=\begin{cases}0 & \text{if } i=0\\ 0 & \text{if } W=0\\ m(i-1,W) & \text{if } w_i>W\\ \max\{m(i-1,W),v_i+m(i-1,W-w_i)\} & \text{otherwise}\end{cases}$$

```
goods = [None, {'w': 4,'v': 8}, {'w': 10,'v': 50}, {'w': 7,'v': 14}, {'w': 11,'v': 44}, {'w
    ': 3,'v': 9}, {'w': 5,'v': 20}, {'w': 14,'v': 28}, {'w': 2,'v': 6}]
# 设置背包最大承重
max_w = 50
# 初始化二维表格m[(i, w)]，将表格中所有价值均初始化为0
# 表示前i个宝物中，最大重量w的组合，所得到的最大价值
# 当i或w为0时，价值为0
m = {(i, w): 0 for i in range(len(goods)) for w in range(max_w + 1)}
# 逐个填写二维表格
# 外层循环为 i个宝物，[1,9)的循环
# 内层循环为 重量w，[1, max_w+1)的循环
for i in range(1, len(goods)):
    for w in range(1, max_w + 1):
```

```
        if goods[i]['w'] > w:
            # 装不下第i个宝物，即不装第i个宝物
            m[(i, w)] = m[(i - 1, w)]
        else:
            # 装得下第i个宝物时，在不装第i个宝物与装第i个宝物这两种情况下，取最大价值
            m[(i, w)] = max(m[(i - 1, w)],
                            m[(i - 1, w - goods[i]['w'])] + goods[i]['v'])
# 输出结果
print(m[(len(goods) - 1, max_w)])
```

6.10 回溯法

6.10.1 回溯法算法思想

回溯法 (探索与回溯法) 是一种选优搜索法，按选优条件向前搜索，以达到目标。但当探索到某一步时，发现原先选择并不优或达不到目标，就退回一步重新选择，这种走不通就退回再走的技术为回溯法，而满足回溯条件的某个状态的点称为“回溯点”。

- 有许多问题，当需要找出它的解集（全部解）或者要求回答什么解是满足某些约束条件的最优解时，往往要使用回溯法。
- 回溯法的基本做法是搜索或者有的组织穷尽搜索。它能避免搜索所有的可能性。即避免不必要的搜索。这种方法适用于解一些组合数相当大的问题。
- 回溯法在问题的解空间树中，按深度优先策略，从根结点出发搜索解空间树。算法搜索至解空间树的任意一点时，先判断该结点是否包含问题的解。如果肯定不包含（剪枝过程），则跳过对该结点为根的子树的搜索，逐层向其祖先结点回溯；否则，进入该子树，继续按深度优先策略搜索。

为了实现回溯，我们先弄明白以下两个问题：

1. 首先应该明确问题的解空间。
2. 其次是组织解空间以便它能用以被搜索到。

6.10.2 问题的解空间和空间树

问题的解空间通常是在搜索问题的解的过程中动态产生的，这是回溯算法的一个重要特性。解空间的确定与我们对问题的描述有关。如何组织解空间的结构会直接影响对问题的求解效率。这是因为回溯方法的基本思想是通过搜索解空间来找到问题所要求的解。对于问题的解空间结构通常以树或图的形式表示，常用的两类典型的解空间树是子集树和排列树。

子集树

当所给的问题是从 n 个元素的集合 S 中找到 S 满足某种性质的子集时，相应的解空间树称为子集树。这类子集树通常有 2^n 个叶结点，遍历子集树的算法需要 $O(2^n)$ 计算时间。

例如，定和子集问题：已知一个正实数的集合 $P = W_1, W_2, \cdots W_n$ 和另一个正实数 M。试求 P 的所有子集 S，使得 S 中的数之和等于 M。

这个问题的解可以表示成 0/1 数组 $x_1, x_2, \cdots, x_n$，依据 W_1 是否属于 S，x_1 分别取值 1 或 0。故解空间中共有 2^n 个元素。它的树结构是一棵完整二叉树。

排列树

当所给问题是确定 n 个元素满足某种性质的排列时，相应的解空间树称为排列树。排列树通常有 n！个叶结点。因此，排列树需要 O(n!) 计算时间。

6.11 算法实例

Code 6.1: 求最大值

```
a = input("请输入a: ")
b = input("请输入b: ")
a = float(a)
b = float(b)
if b < a:
    max = a
else:
    max = b
print(max)
```

Code 6.2: 枚举算法

```
k = int(input("请输入k: "))
n = 10000
while n <= 30000:
    sub1 = int(n / 100)
    sub2 = int(n / 10) % 1000
    sub3 = n % 1000
    if sub1 % k == 0 and sub2 % k == 0 and sub3 % k == 0:
        print(n)
        n = n + 1
    else:
        n = n + 1
```

Code 6.3: 谷角猜想

```
n = int(input("请输入n: "))
while n != 1:
    if n % 2 == 0:
        n = n / 2
    else:
        n = 3 * n + 1
print("猜想成功")
```

Code 6.4: 求平方根

```
a = float(input("请输入a: "))
x0 = float(input("请输入x0: "))
x1 = 0.5 * (x0 + a / x0)
while abs(x1 - x0) > 0.000001:
    x0 = x1
    x1 = 0.5 * (x0 + a / x0)
print(x1)
```

Code 6.5: 斐波那契数列

```
n = int(input("请输入n: "))

def fibonacci(n):
    if n <= 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(n))

def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)

print(f(n))
```

Code 6.6: 跳台阶问题

```
n = int(input("请输入n: "))

def f(n):
    if n <= 2:
        return n
    return f(n - 1) + f(n - 2)

print(f(n))
```

Code 6.7: 二分搜索

```
lis = [1, 3, 4, 6, 7, 8, 10, 13, 14, 20, 22, 24, 30, 50, 80, 98, 100]

n = int(input('输入要查找的数:'))
left = 0
right = len(lis) - 1

while left <= right:
    mid = (left + right) // 2 # 整除
    if n > lis[mid]:
        left = mid + 1
    elif n < lis[mid]:
        right = mid - 1
    else:
        print("要查找的数在数组中的索引为%s" % mid)
        break
else:
    print("没有这个数据")
```

Code 6.8: 阶乘

```
n = int(input("请输入n: "))

def f(n):
    if n == 0:
        return 1
    return n * f(n - 1)

print(f(n))
```